YASH PANDEY
12214802722
CSE - Shift-1 (2nd Year - 2023)
MAIT, ROHINI
DATA STRUCTURES - (2023-24) - 3rd Sem
(Complete file)

Data Structures Lab

CIC-255

Faculty Name : Mr. Ajay Tiwari

Student Name : Yash Pandey

Roll No : 12214802722

Semester : 3rd

Maharaja Agrasen Institute of Technology, PSP Area,

Sector – 22, Rohini, New Delhi - 110086

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Computer Science & Engineering Department

VISION

"To be centre of excellence in education, research and technology transfer in the field of computer engineering and promote entrepreneurship and ethical values."

MISSION

To foster an open, multidisciplinary and highly collaborative research environment for producing world-class engineers capable of providing innovative solutions to real-life problems and fulfil societal needs.

Lab Assessment Sheet

S.No	Experiment	Marks					Total Marks	Date	Signature
		R1	R2	R2	R4	R5			
1.	WAP for Linear And Binary Search							29/8/23	12/9/23
2.	WAP To Implement Sparse Matrix in C++							12/9/23	19/9/23
3.	WAP To Implement Stacks Using Arrays							19/9/23	26/9/23
4.	WAP To Implement Queues Using Arrays							19/9/23	
5.	WAP To Implement Linked List Using Classes Of C++ and							26/9/23	3/10/23
	Perform Insertion Of Node at Start, End And Specified Pos. in LL								
6.	WAP To Implement Doubly-LL and Perform Addition at Start & Deletion at End.							3/10/23	10/10/23
7.	WAP To Implement Circular-LL & Perform Operations on it.							10/10/23	17/10/23
8.	WAP To Perform Stack using LL & Perform Operations on it.							17/10/23	
9.	WAP To Implement Queue using LL & Perform Operations.							21/11/23	

Scanned with CamScanner

S.No	Experiment	Marks					Total Marks	Date	Signature
		R1	R2	R2	R4	R5			
10.	WAP To Implement Binary Tree and perform Traversals.							21/11/23	
11.	WAP To Implement BST and perform Traversal on It.							28/11/23	
12.	Implement different Sorting Algorithm On Array in C++							28/11/23	
	Bubble, Selection, Insersion, Quick, Merge, Heap,								
	Radix, Bucket, Shell and Exchange Sort On Arrays								

Scanned with CamScanner

Lab Assessment Sheet

S.No	Experiment	Marks					Total Marks	Date	Signature
		R1	R2	R2	R4	R5			

S.No	Experiment	Marks					Total Marks	Date	Signature
		R1	R2	R2	R4	R5			

Program-01.

Q :- WAP To Perform Linear & Binary Search On An Array ?

(i) Linear Search On Array

1. It's a sequential search which is done over all items .
2. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection .

<Psuedo Code For Linear Search>

```
procedure linear_search (list, value)
        for each item in the list
        if (match item == value)
                return the item's location
                end if
                end for
                end procedure
```

(ii) Binary Search On Array

1. This search works on the principle of divide and conquer.
2. For this, the data collection should be in the sorted form.
3. Binary search looks for a particular item by comparing the middle most item of the collection.
4. If a match occurs, then the index of item is returned. If the middle item is greater than the item, then the item is searched in the sub-array to the left of the middle item.
5. Otherwise, the item is searched for in the sub-array to the right of the middle item.
6. This process continues on the sub-array as well until the size of the sub array reduces to zero.

<Psuedo Code For Binary Search>

```
BinarySearch(A[0,1,..(N-1)] , value) {
        low = 0 , high = N - 1
        while (low <= high){
            // invariants
            value > A[i] for all i < low ;
            value < A[i] for all i > high ;
            mid = (low + high) / 2;
            if (A[mid] > value) {  high = mid – 1 ;  }
            else if (A[mid] < value){   low = mid + 1  }
            else { return mid;  }
            return not_found;
    }
```

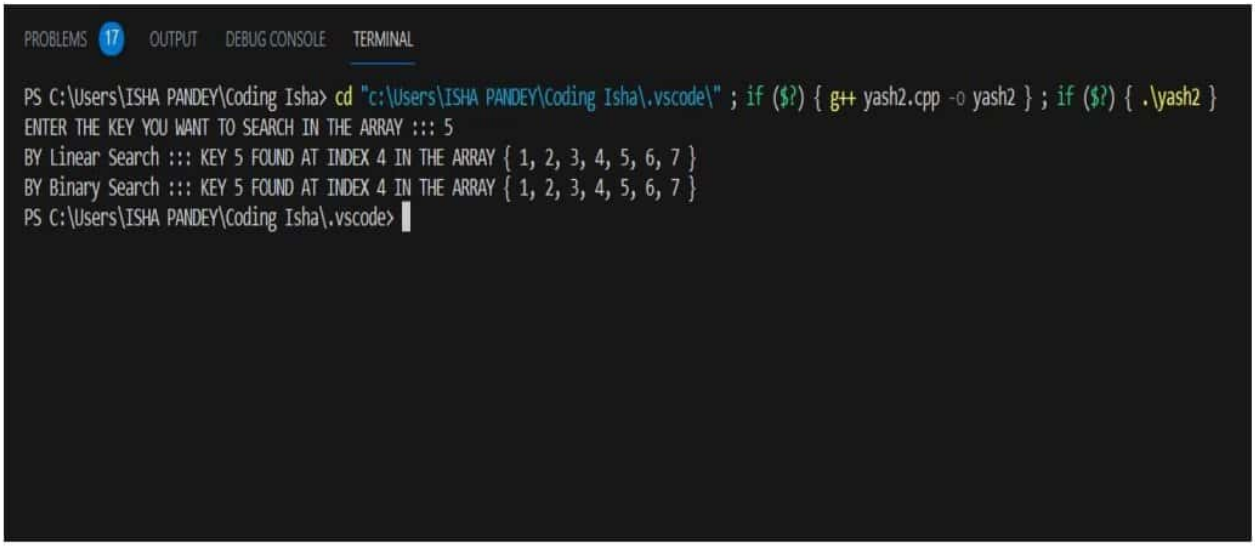
Code For Linear And Binary Search : -

```
#include<iostream>
using namespace std;

void PrintArray(int arr[] , int size);  //Function Prototype……
void LinearSearch(int arr[] , int size , int Key){
    for(int i=0 ; i < size ; i++){
        if(arr[i] == Key){
            cout<<"BY Linear Search ::: KEY ";
            cout<<Key<<" FOUND AT INDEX "<<i<<" IN THE ARRAY ";
            PrintArray(arr , size);
            return;
```

```
                }
            }
            cout<<"BY Linear Search ::: KEY ";
            cout<<Key<<" NOT FOUND IN THE ARRAY ";
            PrintArray(arr , size);
}
void BinarySearch(int arr[] , int size , int Key){
        int start=0 , end=size-1;
        while(start <= end){
                int mid=(start + end)/2;
                if(arr[mid] == Key){
                        cout<<"BY Binary Search ::: KEY ";
                        cout<<Key<<"FOUND AT INDEX"<<mid<<"IN ARRAY";
                        PrintArray(arr , size);
                        return;
                }
                if(arr[mid] > Key){  end = mid-1;  }
                else {  start = mid+1;  }
         }
        cout<<"BY Binary Search ::: KEY";
        cout<<Key<<"NOT FOUND IN THE ARRAY ";
        PrintArray(arr , size);
}
```

```
void PrintArray(int arr[],int size){
        cout<<"{ ";
        for(int i=0 ; i<size ; i++){
            if (i == (size-1) ) {  cout<<arr[i]<<" }"<<endl ;   }
            else {  cout<<arr[i]<<", " ;  }
        }
}

int main() {
        int key;
        int arr[7]={1,2,3,4,5,6,7};
        cout<<"ENTER THE KEY YOU WANT TO SEARCH IN THE ARRAY ::: ";
        cin>>key;
        LinearSearch(arr,7,key);
        BinarySearch(arr,7,key);
        return 0;
}
```

OUTPUT



Viva-Questions :-

Q :- The sequential search, also known as ?

Sol :- The sequential search, also known as Linear Search.

Q :- What is the Primary Req. for the implementation of binary search in an array?

Sol :- The primary requirement for implementing Binary search in an array is that the array must be sorted. The Binary search Algorithm is based on the Divide & Conquer Technique, which means that it will divide the array In Sub-Array.

Q :- Is Binary Search Applicable on Array and LinkedList ?

Sol :- Yes , Binary Search Can Be Applied On Array And Linked List If They Are Sorted.

Q :- What is the Principle of working of Binary search?

Sol :- Binary Search Is Based On The Principle Of Divide And Conquer In Which We Divide The Array In Sub-Array.

Q :- Which Searching Algorithm is Efficient one?

Sol :- Binary Search Is Efficient Method As Compared To Linear Search as The Time Complexity Of Linear Search Is O(n) Whereas , In Binary Search , Time Complexity Is O(Logn) .

Program-02.

Q :- WAP To Implement Sparse Matrix using Arrays ?

(i) Sparse Matrix : -

1. A Matrix is a Two-Dimensional data object made of m rows and n columns, thus , having total m x n values.
2. If most of the elements of the matrix have 0 value, then it is called a Sparse Matrix.

Advantages Of Sparse Matrix Are : -

1. **Storage :** There are lesser non-zero elements than zeros and thus lesser memory can be used to store only those elements.
2. **Computing time :** Computing time can be saved by logically designing a data structure traversing only nonzero elements.

<Psuedo Code For Sparse Matrix Using Array>

```
int main() {
        // Assume 4x5 sparse matrix
        int sparseMatrix[4][5] = { {0 , 0 , 3 , 0 , 4 }, {0 , 0 , 5 , 7 , 0 }, {0 , 0 , 0 , 0 , 0 }, {0 , 2 , 6 , 0 , 0 } };
        int size = 0;
        for (int i = 0 ;  i  <  4 ;  i++){
                for (int j = 0 ;  j < 5 ;  j++){
                        if (sparseMatrix[i][j] !=  0)   {  size++;  }
                                // number of columns in compactMatrix (size) must be ...
                                // equal to number of non - zero elements in sparseMatrix
                        }
                }
        }
```

```
int compactMatrix[3][size];     // Making of new Matrix
        int k = 0;
        for (int i = 0 ;  i  <  4 ;  i++) {
                for (int j = 0 ;  j < 5 ;  j++){
                        if (sparseMatrix[i][j] !=  0) {
                                compactMatrix [0][k]  =  i ;
                                compactMatrix [1][k]  =  j ;
                                compactMatrix [2][k] = sparseMatrix [i][j];
                                k++;
                        }
                }
        }
        for (int i=0 ;  i < 3 ; i++){
                for(int j=0 ; j < size ; j++){    cout<< compactMatrix [i][j]<<" ";     }
                cout<<endl;
        }
```

Code For Sparse Matrix Using Arrays: -

```
#include<iostream>
using namespace std;

int main(){
        int SparseMatrix[5][5] = { {0,0,3,5,0} , {0,4,0,6,0} , {3,0,0,7,0} ,
                                   {0,6,9,0,0} , {2,0,3,0,0} };
        int NonZero_Count=0;
        cout<<"THE SPARSE MATRIX IS ::: "<<endl;
```

```
for (int i=0 ; i < 5 ; i++) {
        for (int j=0 ; j < 5 ; j++) {
                cout<<SparseMatrix[i][j]<<" ";
                if (SparseMatrix[i][j] != 0){    NonZero_Count++;   }
        }
        cout<<endl;
}
int CompactMatrix [3][NonZero_Count] ;
int idx=0;
for (int i=0 ; i < 5 ; i++) {
        for (int j=0 ; j < 5 ; j++) {
                if (SparseMatrix[i][j] != 0){
                        CompactMatrix [0][idx] = i ;
                        CompactMatrix [1][idx] = j ;
                        CompactMatrix [2][idx] = SparseMatrix[i][j] ;
                        idx++ ;
                }
        }
}
cout<<endl;
cout<<"THE REAL MATRIX IS ::: "<<endl;
```

```
        for (int i=0 ; i < 3 ; i++) {
            for (int j=0 ; j < idx ; j++) {
                if (i==0  && j==0)  {   cout<<"ROW  ---> ";  }
                if (i==1  && j==0)  {   cout<<"COL  ---> ";   }
                if (i==2  && j==0)  {   cout<<"VALUE --> ";  }
                cout<<CompactMatrix[i][j]<<" ";
            }
            cout<<endl;
        }
        return 0;
}
```

OUTPUT : -

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\ISHA PANDEY\Coding Isha> cd "c:\Users\ISHA PANDEY\Coding Isha\" ; if ($?) { g++ yash3.cpp -o yash3 } ; if ($?) { .\yash3 }
THE SPARSE MATRIX IS :::
0 0 3 5 0
0 4 0 6 0
3 0 0 7 0
0 6 9 0 0
2 0 3 0 0

THE REAL MATRIX IS :::
ROW  ---> 0 0 1 1 2 2 3 3 4 4
COL  ---> 2 3 1 3 0 3 1 2 0 2
VALUE --> 3 5 4 6 3 7 6 9 2 3


PS C:\Users\ISHA PANDEY\Coding Isha>
```

⇒ Viva - Question of Program - 2.

Q→ What is a Sparse Matrix ?

→ A Sparse Matrix is an Special Matrix in which the Number of zero Elements is Higher Than the No. of Non-Zero Elements.

Q→ What are the different Ways To Represent Sparse Matrix in Memory ?

→ Sparse Matrix Can be Represented in Various Ways as:-

1. Using Arrays ⟶ A 2D-Array having 3 Row i.e. Row, Column and Value of Element and an Column for Each Non-Zero Value in the Given Sparse Matrix.
2. Using Linked List ⟶ A Node Can be defined with 4 Fields in the Node as Row, Column, Value and Pointer To Next Node.
3. Using Dictionary ⟶ A Dictionary Containing Row and Column Number as Key and Value as Matrix Values.

Q→ What is the Advantage of Representing only Non-Zero Element in Sparse Matrix ?

→ A Sparse Matrix is an Matrix in which majority of the Elements are Zero. So, Instead of Storing These Non-Zero Elements and Zero Valued Elements Combined in the Memory, We only Store Non-Zero Elements and helps in Saving Memory and Compilation Time.

Q→ 2D-Array Data Structure is used To Implement Sparse Matrix ?

Q→ What are the Various Type of Representation of Matrix ?

→ Matrix Can be Represented as Row Matrix, Column Matrix, Diagonal Matrix, Zero-Matrix, Upper-Triangular Matrix, Lower-Triangular Matrix, Symmetric Matrix and Hermitian Matrix etc.

Program-03.

Q :- WAP To Implement Stacks using Arrays ?

<Psuedo Code For Stacks Using Array>

insertion(a , top , item , max)

if top = max , then , print 'Stack Overflow' & exit

else top = top + 1 and a[top] = item

deletion(a, top, item)

if top = 0 then print Stack underflow & exit

else item = a[top] end top = top – 1 & exit

Display(top , I , a[i])

if top = 0 then print Stack emtpy & exit

else for i= top to 0 print a[i] end exit

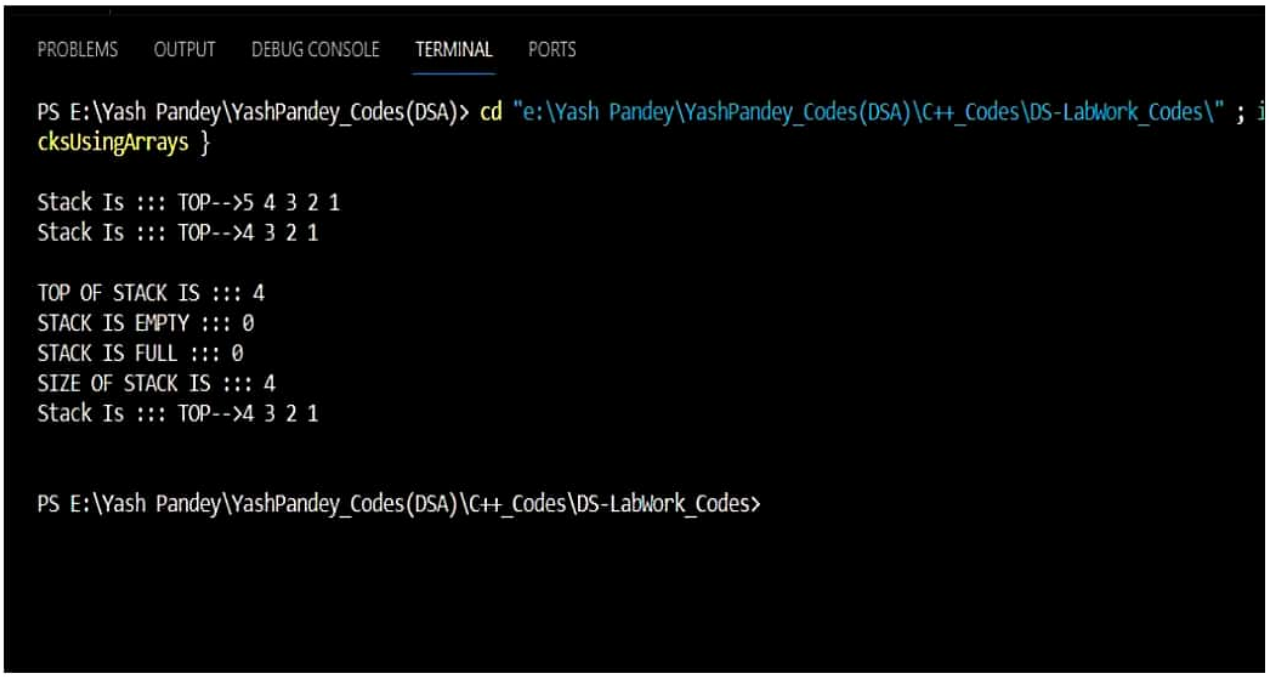
Code For Stacks Using Arrays: -

```
#include<iostream>
using namespace std;

class Stacks{
    private:
        int Capacity;
        int *arr;
        int Top=0;
```

```
public:
        Stacks(int c){
                Capacity=c;
                arr=new int[c];
                Top=-1;
        }
        void push(int data){
                if(Top == Capacity-1){
                        cout<<"OverFlow Condition ::: Stack Is Full"<<endl;
                        return;
                }
                arr[++Top]=data;
        }
        int pop(){
                if(Top == (-1)){
                        cout<<"UnderFlow Condition ::: Stack Is Empty"<<endl;
                        return (-1);
                }
                int temp=arr[Top];
                Top --;
                return temp;
        }
        int peek(){
                if(Top == (-1)){
                        cout<<"UnderFlow Condition ::: Stack Is Empty"<<endl;
                        return (-1) ;
```

```
            }
            return arr[Top];
        }
        void PrintStack(){
            cout<<"Stack Is ::: TOP-->";
            for(int I = Top ; i >=0 ; i --){
                cout<<arr[i]<<" ";
            }
            cout<<endl;
        }
        int size(){     return (Top+1);   }
        bool isEmpty(){    return Top == (-1);     }
        bool isFull(){      return Top ==  (Capacity-1);     }
};
int main(){
        Stacks s(5);
        s.push(1);  s.push(2);  s.push(3);  s.push(4);  s.push(5);
        s.PrintStack();
        s.pop();  s.PrintStack();
        cout<<"TOP OF STACK IS ::: "<<s.peek()<<endl;
        cout<<"STACK IS EMPTY ::: "<<s.isEmpty()<<endl;
        cout<<"STACK IS FULL ::: "<<s.isFull()<<endl;
        cout<<"SIZE OF STACK IS ::: "<<s.size()<<endl;
        s.PrintStack();
        return 0;
}
```

OUTPUT : -



Program-04.

Q :- WAP To Implement Queues using Arrays ?

<Psuedo Code For Queues Using Array>

```
void insert() {
        int num ; scanf( " %d " , &num);
        // Step 1 if(rear==MAX-1) { printf("\n Overflow"); }
        // Step 2 if(front==-1 && rear==-1) { front=rear=0; }
        // Step 3 else { rear++; }
        // Step 4 queue[rear]=num ;
}
int delete() {
        int val; scanf("%d",&val);
        // Step 1 if(front == -1 || front>rear) { printf("Underflow"); }
        // Step 2 else { front++ ; val=queue[front] ; return val; }
}
```

Code For Queues Using Arrays: -

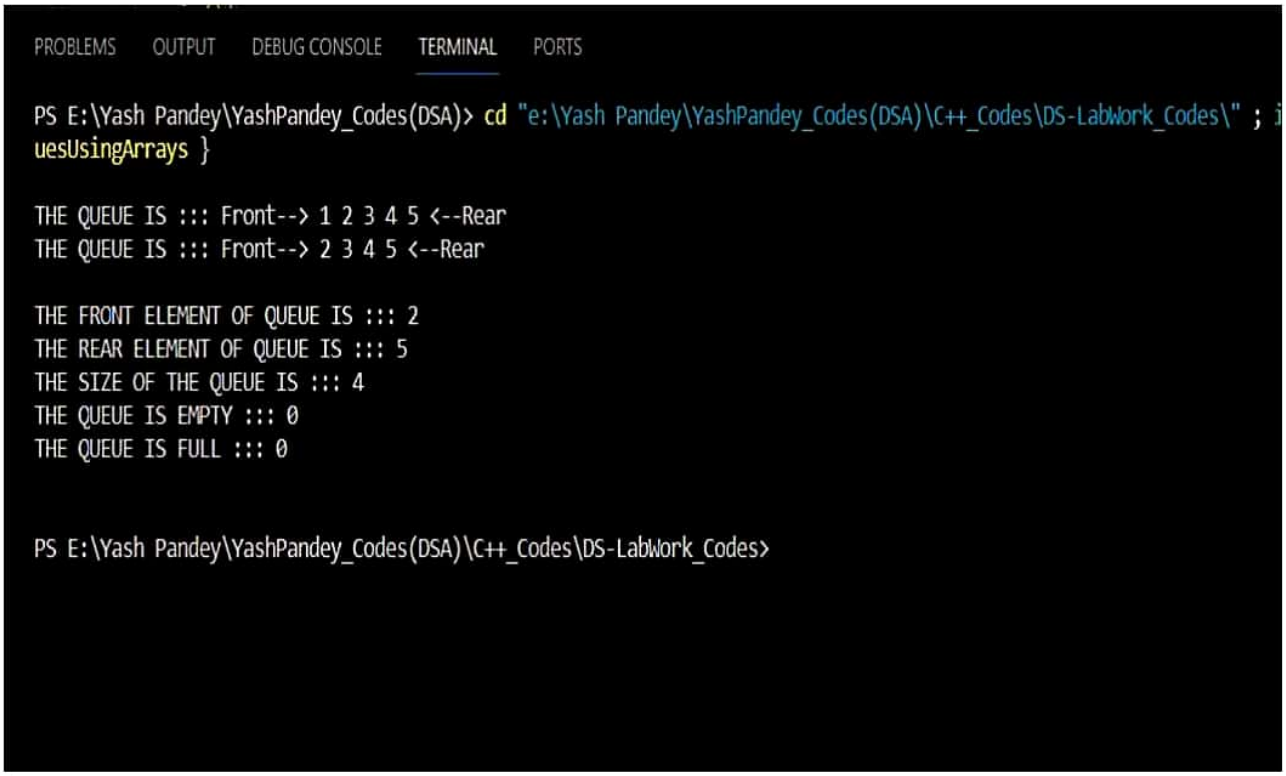
```
#include <iostream>
using namespace std;

class Queues{
        private :
                int * arr;
                int Capacity;
                int Front,Rear;
        public:
```

```
Queues(int c){
        Capacity=c;
        arr=new int[Capacity];
        Front=0;
        Rear=-1;
}
void enqueue(int x){
        if (Rear == Capacity-1){
                cout<<"STACK OVERFLOW ::: STACK IS FULL"<<endl;
                return;
        }
        arr[ ++ (Rear) ] = x;
}
int dequeue(){
        if (Front == (-1) || Front>Rear){
                cout<<"STACK UNDERFLOW ::: STACK IS EMPTY"<<endl;
                return -1;
        }
        int temp=arr[ Front ++ ];
        return temp;
}
int FrontPeek(){
        if (Front == (-1) || Front>Rear){
                cout<<"STACK UNDERFLOW ::: STACK IS EMPTY"<<endl;
                return -1;
        }
```

```
        return arr[Front];
    }
    int LastPeek(){
        if (Front == (-1) || Front>Rear){
            cout<<"STACK UNDERFLOW ::: STACK IS EMPTY"<<endl;
            return -1;
        }
        return arr[Rear];
    }
    int size(){    return (Rear-Front+1);  }
    bool isEmpty (){   return (Front == (-1) || Front>Rear) ? true : false;    }
    bool isFull(){   return (Front==(-1) && Rear==(Capacity-1));  }
    void PrintQueue(){
        cout<<"THE QUEUE IS ::: Front--> ";
        for(int i = Front ; i <= Rear ; i++){
            cout<<arr[i]<<" ";
        }
        cout<<"<--Rear"<<endl;
    }
};
int main(){
    Queues q(5);
    q.enqueue(1);  q.enqueue(2);  q.enqueue(3);
    q.enqueue(4);  q.enqueue(5);
    q.PrintQueue();
    q.dequeue(); q.PrintQueue();
```

```
    cout<<"THE FRONT ELEMENT OF QUEUE IS ::: "<<q.FrontPeek()<<endl;
    cout<<"THE REAR ELEMENT OF QUEUE IS ::: "<<q.LastPeek()<<endl;
    cout<<"THE SIZE OF THE QUEUE IS ::: "<<q.size()<<endl;
    cout<<"THE QUEUE IS EMPTY ::: "<<q.isEmpty()<<endl;
    cout<<"THE QUEUE IS FULL ::: "<<q.isFull()<<endl;
    return 0;
}
```

OUTPUT : -



Topic..... Viva Questions _ 3. .....Date..19-09-2023

→ Viva Question Based on Program 3 & 4...

Q→ Write A Program To Sort an Queue ?

→
```
#include <iostream>
using namespace std;

Class Queues {
    private :
        int arr[5] = {0};
        int Front, Rear;
    public :
        Queues (Void) {
            Front = 0 ; Rear = -1;
        }
        Void enqueue (int x) { // Enqueue Fn For Queue.....
            if ( Front == 5) { return ; }
            arr [++ Rear] = (x);
        }
        int dequeue () { // Dequeue Fn in Queue.....
            if ( Front == (-1) || Front > Rear ) { return (-1)}
            int temp = arr [Front ++];
            return temp;
        }
        Void SortQueue () {
            for (int i = 0 ; i < 4 ; i++) { // Selection Sort.....
                for (int j = (i+1) ; j < 5 ; j++) {
                    int temp ;
                    if ( arr [i] > arr [j]) {
                        temp = arr [i] ; arr [i] = arr [j];
                        arr [j] = temp;
                    }
                }
            }
```

{return (-1);}

Teacher's Sign .....

Topic.....Date.....

```
void PrintQueue() {
        Cout << " The Queue is ::: Front -> ";
        for (int i = Front ; i <= Rear ; i++){
                Cout << arr[i] << "  ";
        }
        Cout << " <- Rear " << endl;
    }
};
int main () {
        Queues q ();
        q.enqueue (2); q.enqueue (3); q.enqueue (1);
        q.enqueue (5); q.enqueue (4);
        q.PrintQueue ();
        q.SortQueue ();
        q.PrintQueue ();
        Return 0;
}
```

∴ OUTPUT ⟶ The Queue is ::: Front -> 2 3 1 5 4 <- Rear
The Queue is ::: Front -> 1 2 3 4 5 <- Rear

Q ⟶ How Queue Differs from Stack?

⟶ Queues Follows "First In First Out (FIFO)" operation and the Insertion in Queues are From the Rear and the deletion of Element from the Front of Queue, whereas, In Stacks, "Last In First Out (LIFO)" operation is followed in Stacks and Insertion as well as Deletion In case of Stacks are done from Top of Stack only. In Queues Implementation Using Arrays, We Require 2 Pointers as Front and Rear whereas in case of Stacks, We can Implement Stack with only one Pointer "Top".

⟶ Stacks are Visualized as Vertical Collections whereas Queues are Visualized as Horizontal Collection in the Memory.

Teacher's Sign .....

Topic.....Date.....

Q → Write a Program To Reverse a Queue?

→
```
#include <iostream>
using namespace std;

Class Queues {
    private :
        int arr[5] = {0}
        int Front, Rear;
    public :
        Queues (Void) {
            Front = 0 ; Rear = (-1);
        }
        Void enqueue (int data) { // Enqueue Fn For Queue.....
            if (Front == 5 ) { return ; }
            arr [ ++ Rear ] = data;
        }
        int dequeue ( ) {  // Dequeue Fn for Queue....
            if (Front == (-1) || Front > Rear ) {  return (-1); }
            int temp = arr [Front ++ ];
            return temp;
        }
        Void ReverseQueue () {  // Fn To Reverse Queue.....
            int n = (Rear - Front + 1 ) ;
            for (int i=0 ; i< (n/2) ; i++) {
                int temp = arr [i] ;
                arr [i] = arr [n-i-1];
                arr [n-i-1] = temp;
            }
    }
```

"P. T. O"

Teacher's Sign .....

Topic.....Date.....

```
    void PrintQueue() {
        cout << "The Queue is ::: Front -> ";
        for (int i = Front; i <= Rear; i++) {
            cout << arr[i] << " ";
        }
        cout << " <- Rear " << endl;
    }
};
int main() {
    Queues q();
    q.enqueue(1); q.enqueue(2); q.enqueue(3);
    q.enqueue(4); q.enqueue(5);
    q.PrintQueue();
    q.ReverseQueue();
    q.PrintQueue();
    return 0;
}
```

∴ "OUTPUT" → The Queue is ::: Front → 1 2 3 4 5 ← Rear
The Queue is ::: Front → 5 4 3 2 1 ← Rear

Q→ What is Stack? How much memory is been allocated To Stack? Why Stack is called as an Abstract Data Type?

→ Stack is an Linear Data Structure which follows "LIFO" order for The Insersion and deletion of Elements. Stack Implementation is done with Pointer 'Top' and Insersion / Deletion are done in Stack From Top.

→ The Amount of Memory Allocated To Stack is 1 To 8 MB. The Size of The Stack depends on The Operating System and The Language of Compilers Used.

→ A Stack is an Abstract Datatype (ADT) because it is abstract in Both its Implementation and Interface. In Stack, There are mainly 2 Operations as Push() and Pop(). Stack follows 'LIFO' Operation in Implementation.

Teacher's Sign.....

Program-05.

Q :- WAP To Implement Insersion In Linked-List At : -

1. Insert New Node At A Specified Position In LinkedList .
2. Delete Node At A Specified Position In LinkedList .
3. Write Function For Reversal Of LinkedList

<Psuedo Code For Creation Of LinkedList>

```
struct Node* new_node = new (Node);
new_node->data = new_data;
new_node->next = Null;   head=start ;
while (head->next!=NULL) {     head=head->next;      }
head->next=new_node;
```

<Psuedo Code For Insersion At Specified Position In LL>

```
struct Node* NewNode =new(node);
NewNode->data = new Node( data ) ;
if (prev_node == NULL)   {
        NewNode->next=start;
        Start=NewNode ;
}
NewNode->next = prev_node->next;
prev_node->next = NewNode;
```

<Psuedo Code For Deletion In LinkedList>

```
head=start;
while(head!=null && head->data!= data) {
        Prev_node=head;
        head=head->data;
}
Prev_node->next=head->next;
```

<Psuedo Code For Reversal Of LinkedList>

```
struct Node* prev = NULL;
struct Node* current = *head_ref;
struct Node* next;
while (current != NULL)  {
        next = current->next;   current->next = prev ;
        prev = current;    current = next ;
}
*head_ref = prev;
```

Code For Implementation Of Linked List: -

```
#include<iostream>
using namespace std;
class Node{
    public:
        int data;
        Node* next;
        Node(int CurrData){
            this->data=CurrData;   this->next=NULL;
        }
};
```

```
class LinkedList{
    private:
        int size;
        Node* Head;
        Node* Tail;
    public:
        LinkedList(){
            this->size=0;
            this->Head = this->Tail = NULL;
        }
        void AddFirst(int data){
            Node* NewNode=new Node(data);
            if(Head==NULL){
                Head=Tail=NewNode;    size++;
                return;
            }
            NewNode->next=Head;  Head=NewNode;
            size++;
        }
        void AddLast(int data){
            Node* NewNode=new Node(data);
            if(Head==NULL){
                Head=Tail=NewNode;   size++;
                return;
            }
```

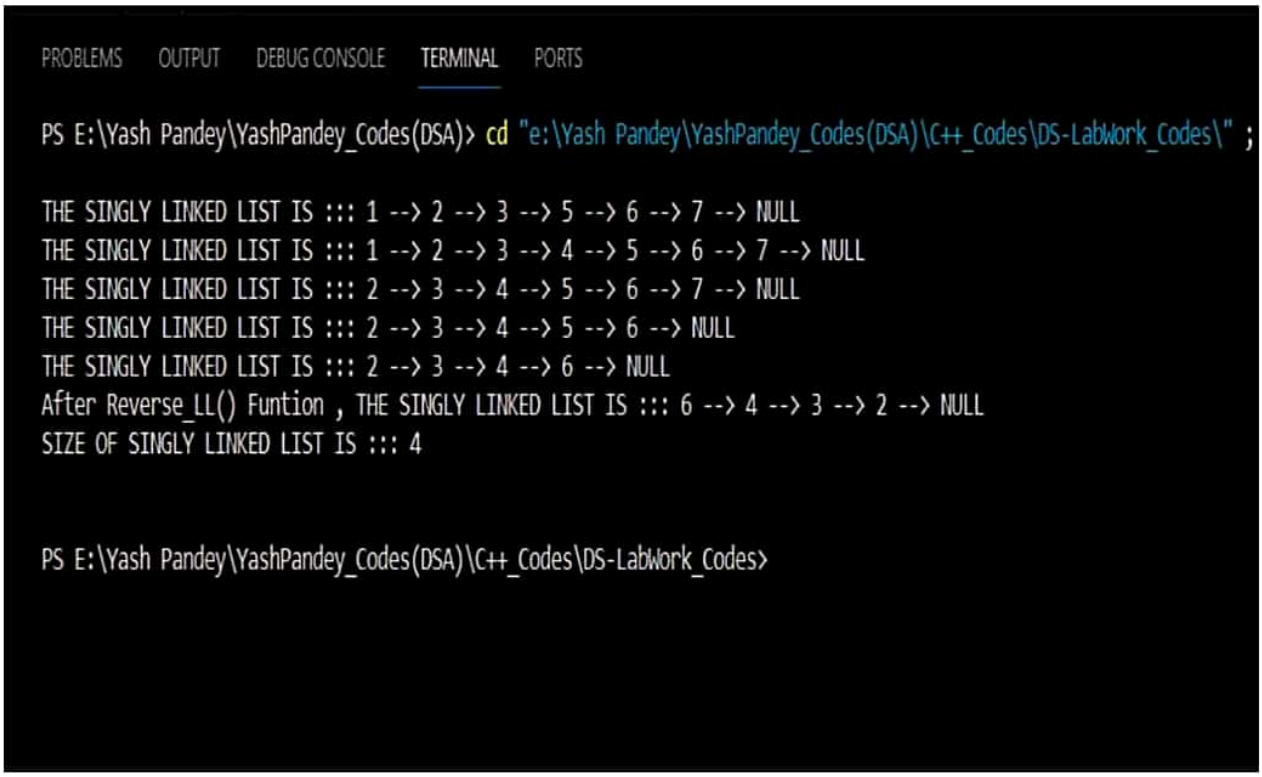
```
        Tail->next=NewNode;    Tail=NewNode;
        size++;
}
void AddMiddle(int data,int idx){
        Node* NewNode=new Node(data);
        if(Head==NULL){   AddFirst(data);   return;    }
        if(idx==size){        AddLast(data);   return;    }
        Node* temp=Head ;    int index=0;
        while(index != (idx-1)){
                temp=temp->next;    index++;
        }
        NewNode->next = temp->next;
        temp->next=NewNode;  size++;
}
int DeleteStart(){
    if (Head==NULL){
        cout<<"LINKEDLIST IS EMPTY ,DELETION NOT POSSIBLE\n";
        return (-1);
    }
        int Value=Head->data;
        Node*temp=Head ;   Head=Head->next;
        free(temp);    size--;
        return Value;
}
```

```
int Delete_End(){
    if(Head==NULL){
        cout<<"LINKEDLIST IS EMPTY ,DELETION NOT POSSIBLE\n";
        return (-1);
    }
        Node*temp=Head;
        while(temp->next->next != NULL){    temp=temp->next;    }
        int Value=ptr->data;
        Node* ptr =temp->next;     ptr->next=NULL;
        Tail = temp->next = NULL ;
        free(ptr);     size--;
        return Value;
}
int Delete_Anywhere(int idx){
        if (idx == 0){
                int Value=DeleteStart();     return Value;
        }
        Node* temp=Head;       int CurrPos=0;
        while(CurrPos != (idx - 1) ){
                temp=temp->next;     CurrPos++;
        }
        int Value=ptr->data;
        Node* ptr =temp->next;     temp->next=ptr->next;
        free(ptr);       size--;
        return Value;
}
```

```
        void Reverse_LL(){
            Node* prev=NULL ;     Node* temp=Head;
            Node* ahead=NULL;
            while(temp ! = NULL){
                ahead=temp->next;   temp->next=prev;
                prev=temp ;     temp=ahead;
            }
            Tail=Head;    Head=prev;
            cout<<"After Reverse_LL() Funtion , ";
        }
        int SizeLL(){   return size;   }
        void PrintLL(){
            Node* temp=Head;
            cout<<"THE SINGLY LINKED LIST IS ::: ";
            while(temp!=NULL){
                cout<<temp->data<<" --> ";
                temp=temp->next;
            } cout<<"NULL"<<endl;
        }
};
int main(){
    LinkedList ll;
    ll.AddFirst(3);   ll.AddFirst(2);   ll.AddFirst(1);
    ll.AddLast(5);    ll.AddLast(6);    ll.AddLast(7);
    ll.PrintLL();     ll.AddMiddle(4,3);     ll.PrintLL();
    ll.DeleteStart();     ll.PrintLL();
```

```
        ll.Delete_End();    ll.PrintLL();
        ll.Delete_Anywhere(3);    ll.PrintLL();
        ll.Reverse_LL();    ll.PrintLL();
        cout<<"SIZE OF SINGLY LINKED LIST IS ::: "<<ll.SizeLL()<<endl;
        return 0;
}
```

OUTPUT : -



Topic..... Viva Question - 5. .....Date..... 26-9-2023

⇒ Viva - Questions Based on Program - 5.

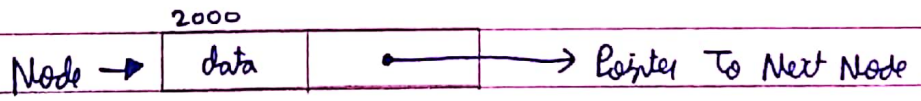
Q→ What Type of Memory Allocation is Referred for Linked Lists?

→ Dynamic Memory Allocation is been Referred for The Linked Where we Allocate The Memory To Linked List during Run Time. Dynamic Memory Allocation (DMA) is an method of requesting memory during The Program Execution. DMA is done with help of malloc(), calloc(), realloc() and free().

→ In malloc(), when memory is Allocated, The Starting Address of Allocated memory is Returned. Also, In malloc(), The Memory is been Initialized with Garbage Value Whereas in Case of calloc(), when memory is Allocated, The Allocated memory is been Assigned with '0' Not Garbage Value. When we Input Value, Then, The default Value '0' is been changed To Passed Value.

Q→ Describe what is Node in Linked List? Name The Type of Linked List?

→ A Node in a Linked List is a data Structure that Contains A Value and an Pointer To The Next Node. The Value is The data which is Stored in The Node and The Pointer is The location To The Next Node of The Linked List. Nodes are Linked Together with Each other which forms a chain where The First Node of This Chain is Called "Head" and The last Node of The Chain is "Tail".



* The 4 Types of Linked List are:-

1. Singly Linked List.
2. Circular Singly Linked List.
3. Doubly Linked List.
4. Circular Doubly Linked List.

Teacher's Sign .....

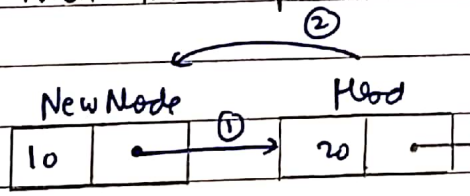
Topic..... Date.....

Q → What are The Difference between Lines Array and Linked List?

→ Array	Linked List
→ Size of Array is Fixed.	→ Size of Linked List is Not Fixed.
→ Memory is Allocated Linearly.	→ Memory is Allocated Dynamically.
→ It is Nessesary to Specify The N.o. of Elements while declaration of Array.	→ It is Not Nessesary To Specify The N.o. of Elements while declaration of Linked List.
→ It occupies less Space than a Linked List with Same Elements.	→ It occupies more memory due to Node which is Consist of Value & Pointer.
→ Deletion of Element in Array is Not Possible.	→ Deletion of Element in Linkedlist is Possible.

Q → Mention The Steps while Insertion of Node at Start of Linked List?

→

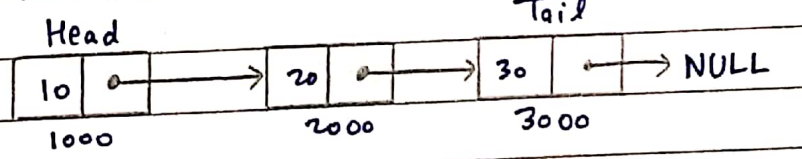


* The Steps Involved are →
- Creation of Node 'New Node' in Memory
- New Node → next = Head ;
- Head = New Node ;

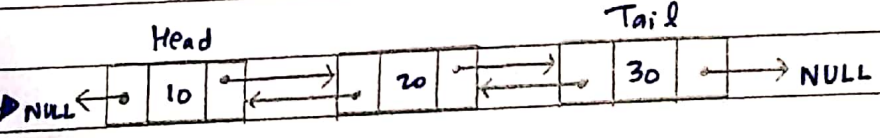
Q → For Which Linked List, you will find The last Node Contains a Null Pointer?

→ For Singly LL and Doubly LL, last Node Contains an Null Pointer.

1. Singly Linked List →



2. Doubly Linked List →



Teacher's Sign .....

Program-06.

Q :- WAP To Implement Doubly-Linked-List And Perform : -

1. Insert New Node At Start Of Doubly-LinkedList .
2. Delete Last Node In Doubly-LinkedList .

<Psuedo Code For Insersion At Start Of Doubly-LinkedList>

```
void push(struct Node** head_ref, int new_data) {
        struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));
        new_node->data = new_data;
        new_node->next = (*head_ref);
        new_node->prev = NULL;
        if ((*head_ref) != NULL)  {    (*head_ref)->prev = new_node;    }
}
```

<Psuedo Code For Deletion Of Last Node In Doubly-LL>

```
void deleteNode(Node** head_ref, Node* del) {
        if (*head_ref == NULL || del == NULL)       return;
        if (*head_ref == del)          *head_ref = del->next;
        if (del->next != NULL)          del->next->prev = del->prev;
        if (del->prev != NULL) del->prev->next = del->next;
```

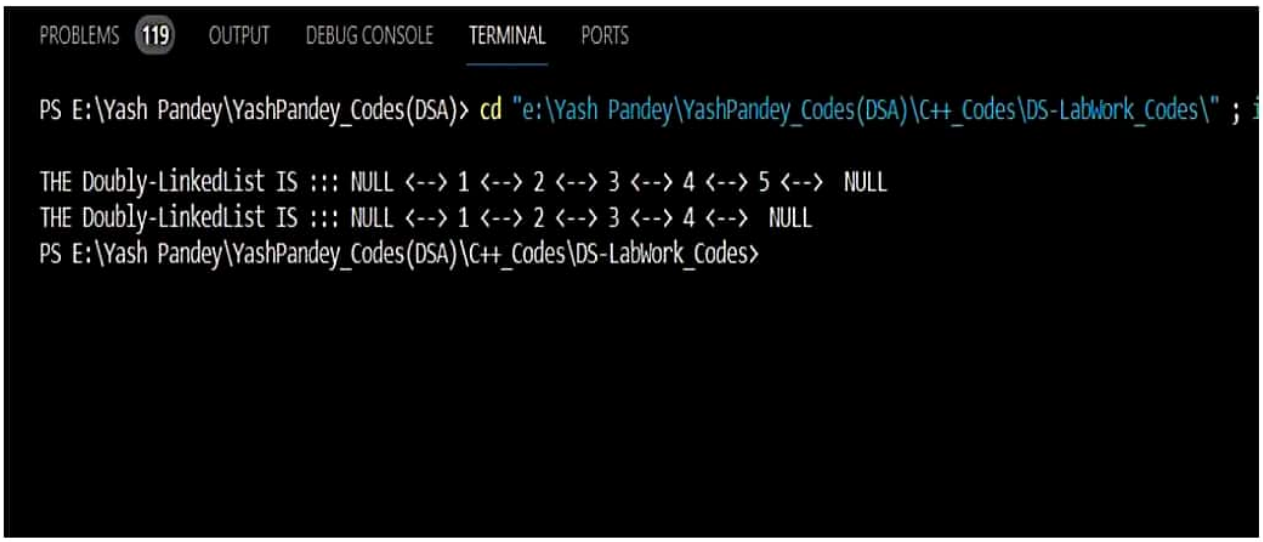
Code For Implementation Of Doubly-Linked List: -

```
#include<iostream>
using namespace std;
```

```
class Node{
        public:
                int data;    Node* prev;  Node* next;
                Node(int CurrData){
                        this->data=CurrData;
                        this->prev=this->next=NULL;
                }
};
class DoublyLL{
        private:
                Node* Head;     Node* Tail;
        public:
                DoublyLL(){     this->Head=this->Tail=NULL;       }
                void AddFront(int data){
                        Node* NewNode=new Node(data);
                        If (Head == NULL){
                                Head=Tail=NewNode;    return;
                        }
                        Head->prev=NewNode;   NewNode->next=Head;
                        Head=NewNode;
                }
                void RemoveLast(){
                        if (Head == NULL){     return ;      }
                        Node* Temp=Tail;   Tail=Tail->prev;
                        Tail->next=Temp->prev=NULL;     free(Temp);
                }
```

```
        void PrintDLL(){
            Node* Temp=Head;
            cout<<"THE Doubly-LinkedList IS ::: NULL <--> ";
            while(Temp!=NULL){
                cout<<Temp->data<<" <--> ";
                Temp=Temp->next;
            }  cout<<" NULL"<<endl;
        }
};

int main(){
   DoublyLL dll;
   dll.AddFront(5);   dll.AddFront(4);   dll.AddFront(3);
   dll.AddFront(2);   dll.AddFront(1);   dll.PrintDLL();
   dll.RemoveLast();   dll.PrintDLL();
   return 0;
}
```

OUTPUT : -



Topic..... Viva - Questions - 6. .....Date..... 3-10-23.....

→ Viva Questions Based On Program - 6.

Q→ What is memory Efficient Doubly Linked List?

→ Memory Efficient Doubly Linked List has only one Pointer to Traverse The Linked List Back and Forth. The Implementation of memory Efficient Doubly Linked List is based on The Concept of Pointer Difference. It Uses Bitwise XOR operator To Store the front and Rear Pointer Addresses. This allows the list to have only one Pointer To Traverse Back and forth on The Doubly Linked List.

Q→ What is The Advantage of Doubly Linked List?

→ Some Advantages of Doubly Linked List are :-

→ Easy Traversal on Both Side of Doubly Linked List.
→ Efficient Insersion & Deletion in Doubly Linked List.
→ Efficient for The Reversal of Doubly Linked List.
→ Efficient for The Insersion at a Known Index.
→ Simplified Algorithms for Doubly Linked List.

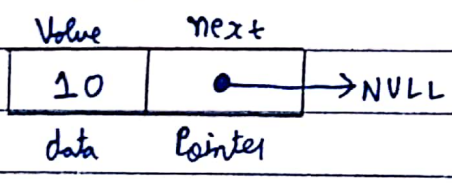
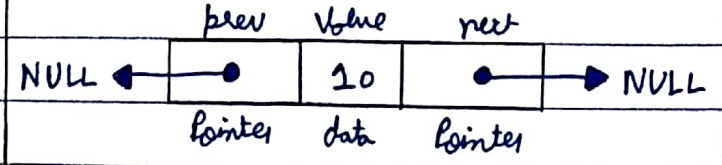
Q→ Why is a Doubly Linkedlist is more Useful Than Singly Linkedlist?

→ Doubly Linked List are more useful Than Singly Linked List because :-

→ Traversal In Doubly LL is more Efficient Than Singly LL.
→ Insersion In DLL is more Efficient Than Singly LL.
→ Simplified Algorithms for Doubly Linked List Than Singly LL.
→ Deletion In Doubly LL is more Efficient Than Singly LL.
→ Reversal of Doubly LL is more Efficient Than Singly LL.

Topic.....Date.....

Q → What is The Difference b/w Linked List & Doubly Linked List?

→	Singly Linked List	Doubly Linked List
→	Nodes in Singly linked list is Consist of data field and next link field i.e. Pointer To Next Node of Linked List.	→ Nodes in Doubly linked list is Consist of data field and Two Pointer i.e. Pointer To Next & Prev. Node of Doubly LL.
→	It allows Traversal in Only One Way.	→ It allows for Two-Way Traversal.
→	It Requires only one Pointer (Head).	→ It Requires Two Pointer (Head & Tail).
→	It occupies less memory.	→ It occupies more memory.
→	Complexity of Insersion & Deletion at Known Position is O(n).	→ Complexity of Insersion And Deletion at Known Position is O(1).
→	Node in Singly LL :- 	→ Node in Doubly LL :- 

Program-07.

Q :- WAP To Implement Circular-Linked-List And Perform : -

1. Insert New Node At Start Of Circular -LinkedList .
2. Delete Last Node In Circular -LinkedList .

<Psuedo Code For Insersion At Start Of Circular-LinkedList>

```
struct Node *  addBegin(struct Node *last, int data) {
        if (last == NULL)      return addToEmpty(last, data);
        struct Node *temp  = (struct Node *)malloc(sizeof(struct Node));
        temp -> data = data;      temp -> next = last -> next;    last -> next = temp;
        return last;
}
```

<Psuedo Code For Deletion Of Last Node In Circular -LL>

```
struct Node *  addBegin(struct Node *last, int data)   {
        if (last == NULL)       return addToEmpty(last, data);
        struct Node *temp = (struct Node *)malloc(sizeof(struct Node));
        if (curr->next == head) { head = NULL;   free(curr);   return;     }
        if (curr == head) {     prev = head;
                while (prev -> next != head)
                        prev = prev -> next;    head = curr->next;
                        prev->next = head;     free(curr);
        }
         else if (curr -> next == head)  {   prev->next = head;   free(curr);    }
        else {   prev->next = curr->next;    free(curr);     }
}
```

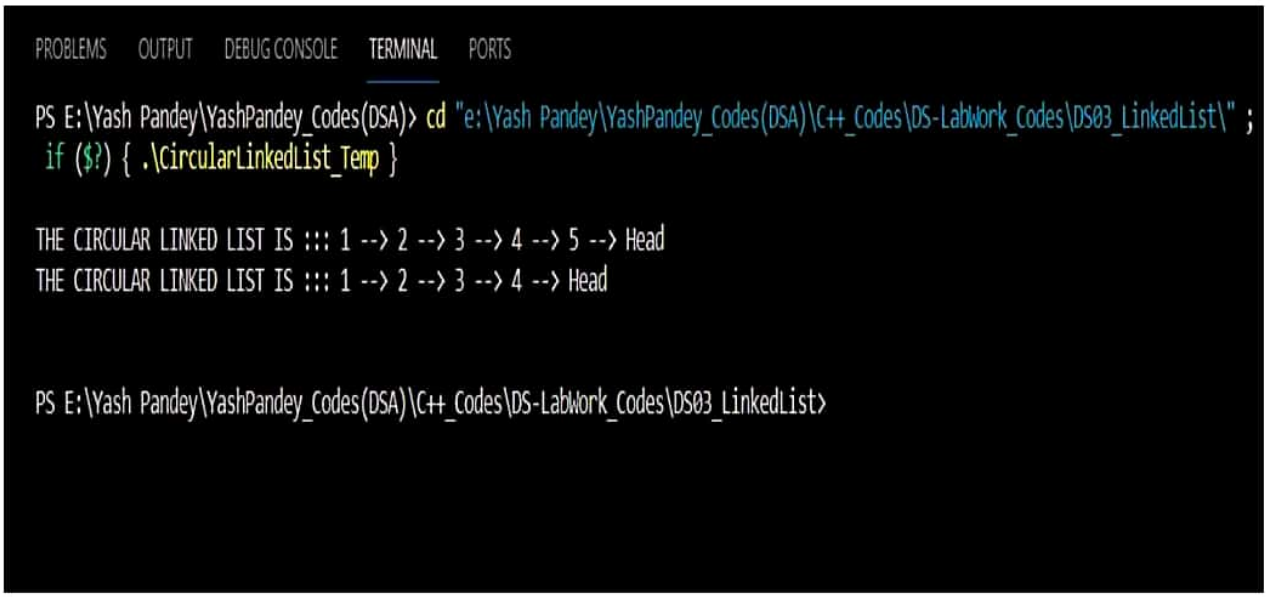
Code For Implementation Of Doubly-Linked List: -

```
#include<iostream>
using namespace std;
class Node{
    public:
        int data;   Node* next;
        Node (int CurrData){
            this->data=CurrData;   this->next=NULL;
        }
};
class CircularLL{
    private:
        Node* Head;   Node* Tail;
    public:
    CircularLL(){
        Head=Tail=NULL;
    }
    void AddFirst(int data){
        Node* NewNode=new Node(data);
        If (Head==NULL){
            Head=Tail=NewNode;   Tail->next=Head;   return;
        }
        NewNode->next=Head;
        Head=NewNode;    Tail->next=Head;
    }
```

```
int RemoveLast(){
    if (Head==NULL){     return (-1);      }
    if (Head->next==NULL){
        int Value=Head->data;     Head=Tail=NULL;     return Value;
    }
    Node* temp=Tail;    int Value=Tail->data;    Node* ptr=Head;
    while(ptr->next!=Tail) {   ptr=ptr->next;    }
    Tail=ptr;    Tail->next=Head;
    free(temp);    return Value;
}
void PrintCLL(){
    Node* temp=Head;
    cout<<"THE CIRCULAR LINKED LIST IS ::: ";
    while (temp != Tail){
        cout<<temp->data<<" --> ";
        temp=temp->next;
    }
    cout<<Tail->data<<" --> Head"<<endl;
}
};
```

```
int main(){
        CircularLL cll;
        cll.AddFirst(5);      cll.AddFirst(4);      cll.AddFirst(3);
        cll.AddFirst(2);      cll.AddFirst(1);      cll.PrintCLL();
        cll.RemoveLast();  cll.PrintCLL();
        return 0;
}
```

OUTPUT : -



Topic: Viva - Questions Date: 10-10-23

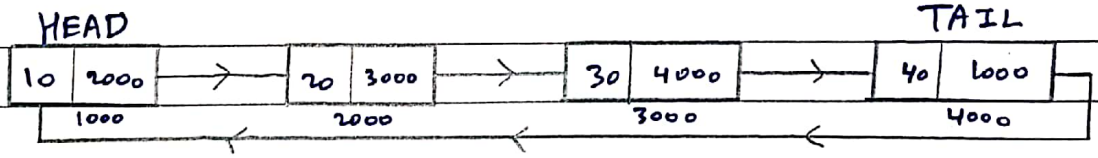
⇒ Viva Questions of Program - 07.

Q → Is There any Null Pointer in a Circular Linked List?

→ No, There is no "NULL" Pointer in an Circular Linked List Because In Case of Circular Linked List, The Head is been Connected by The Tail of The Linked List which Results in The formation of a Loop. Thus, There is No "NULL" Pointer in Case of Circular Linked List.

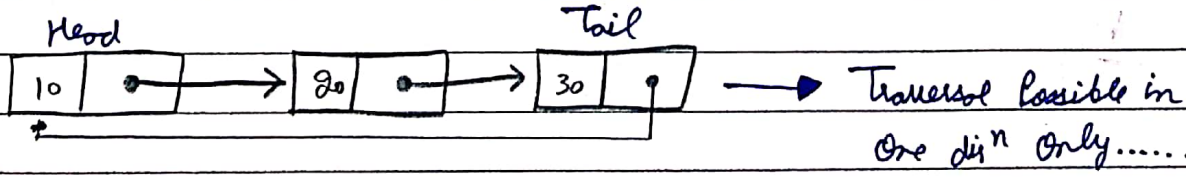
Q → What is a Circular Linked List?

→ A Circular Linked List is a Linked List where all The Nodes are Connected To form a Circle. In a Circular Linked List, The First Node and The last Node are Connected To form a Loop / Circle. There is no "NULL" Pointer at The End of Linked List. In Circular Linked List, The Tail of The Linked List is Connected To The Head of Linked List.



Q → Whether a Circular Linked List is Single Way List or Two Way List?

→ A Circular Linked List is only a Single Way List Because Traversal is only possible from Head → Tail, Reverse Traversal from Tail to Head of Linked List via Nodes is Not Possible.



"On Next Page"

Teacher's Sign .....

Topic.....Date.....

Q → Which pointer signifies the starting of circular linked list?

→ The Head pointer signifies the starting of the circular linked list where the traversal on the circular linked starts from the Head of the circular linked list. Thus, Head pointer signifes the starting of circular linked list.

Q → Describe the steps to delete the starting Node of linked list?

→ The 3 cases possible are :-

Case I → When the circular linked list is Empty, Then, Deletion Not possible.

Case II → When the circular linked list has only one Node, Then, Simply make the Head & Tail of that circular linked list, free / delete the single Node a Return the value of that Node.

Case - III → When Node > 1, Then, Eirstly move Head to the Next Node, Point the 1st Node with some pointer "temp", Move the pointer of Tail to new Head Make the Next of 'temp' Node as NULL, delete the Node 'temp' and Return the value of the Temp Node. The steps are :-

① Node * Temp = Head;
② Head = Head → next;
③ Tail → next = Head;
④ delete (Temp);
⑤ return Temp → data;

★ Time Complexity To delete the Eirst Node of CLL will be O(1).

Teacher's Sign .....

Program-08.

Q :- WAP To Implement Stacks Using Linked-List And Perform Push , Pop , Peek & Traverse The Stack ?

< Psuedo Code For Insersion At Top Of Stack >

```
void push(int data)  {
        struct Node* temp = new Node();
        if ( ! temp) {    cout << " \n Heap Overflow";    }
        temp->data = data;
        temp->link = top;
        top = temp;
}
```

<Psuedo Code For Deletion From Top Of Stack>

```
void pop() {
        struct Node* temp;
        if (top == NULL) {     cout << "\nStack Underflow" << endl;     }
        else {
                temp = top;
                top = top->link;
                temp->link = NULL;
                free(temp);
        }
}
```

Code For Implementation Of Stack Using Linked List: -

```
#include<iostream>
using namespace std;
class Node{
  public:
      int data;      Node* next;
      Node(int CurrData){
            this->data=CurrData;      this->next=NULL;
      }
};
class Stacks_LinkedList{
      private:
            Node* head;        int size , Capacity;
      public:
            Stacks_LinkedList(int c){
                  this->Capacity=c;   this->size=0;
                  this->head=NULL;
            }
            void push(int data){
                  if(this->size==this->Capacity){          return;        }
                  Node* NewNode=new Node(data);
                  NewNode->next=this->head;
                  this->head=NewNode;
                  this->size++;
            }
```

```
        int pop(){
            if(this->head==NULL){     return -1;     }
            Node* temp=this->head; int Value=temp->data;
            this->head=this->head->next;    temp->next=NULL;
            this->size--;           free(temp);
            return Value;
        }
        int peek(){
            if(this->head==NULL){     return -1;              }
            return this->head->data;
        }
        void PrintStack(){
            cout<<"THE STACK IS ::: TOP--> ";
            Node* temp=head;
            while(temp != NULL){
                cout<<temp->data<<" ";
                temp=temp->next;
            }
        }
        int Size(){  return this->size;  }
        bool isEmpty(){  return this->head==NULL;  }
        bool isFull(){  return this->size==this->Capacity;  }
};
```

```
int main(){
        Stacks_LinkedList sll(10);
        sll.push(1);        sll.push(2);        sll.push(3);
        sll.push(4);        sll.push(5);        sll.push(6);
        sll.PrintStack();       sll.pop();      sll.PrintStack();
        cout<<"TOP OF STACK IS ::: "<<sll.peek()<<endl;
        return 0;
}
```

OUTPUT : -

Topic..... Viva – Questions ..... Date..... 17-10-23

⇒ Viva Questions of Program - 08.

Q→ The Following postfix expression with a Single Digit Operands is Evaluated as :-

8 2 3 ^ / 2 3 * + 5 1 * -

⊙ ^ is Exponential Operator. The Top Two Elements of The Stack after The first - (*) is Evaluated are :-

(A) 6, 1
(B) 5, 7
(C) 3, 2
(D) 1, 5

→ (C) Part is Correct.

Q→ What is The Best Case Time Complexity of deleting a Node in LL?

(A) O(n)
(B) O(log n)
(C) O(n log n)
(D) O(1)

→ (D) Part is Correct.

Q→ Which of The Statement are Not Correct wrt Singly-LL & Doubly-LL :-

(A) Insersion & Deletion at Known Position in SLL is O(n) & DLL is O(1).
(B) SLL Uses less Memory per Node Than Doubly-LL.
(C) DLL has more Searching Power Than SLL.
(D) N.O. of Node fields in SLL are More Than DLL.

→ (A) & (D) are Correct.

P.T.O

Teacher's Sign .....

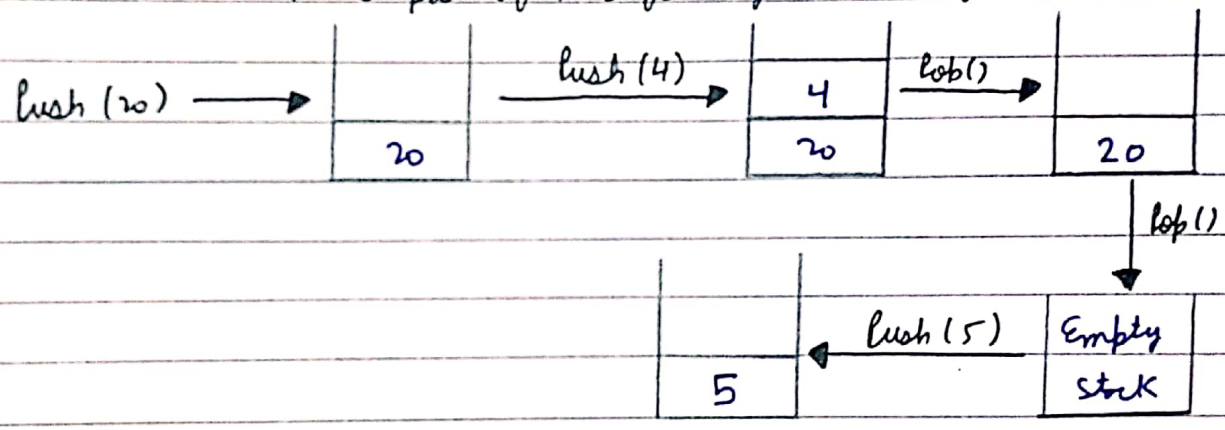
Topic..... Date.....

Q → What does "Stack overflow" refer To :-

(A) Accessing Item from a Undefined Stack.
(B) Adding Items To a full Stack.
(C) Removing Items from an Empty Stack.
(D) Index out of Bounds Exception.

→ (B) Part is Correct.

Q → What will be The output after performing below sequence :-



→ The Top of Stack after above operations is :-

(A) 20 (C) 5
(B) Stock Overflow (D) NULL

→ (C) Part is Correct.

Teacher's Sign .....

Program-09.

Q :- WAP To Implement Queues Using Linked-List And Perform Insert, Delete & Traverse The Queue ?

< Psuedo Code For Insersion At Rear Of Queue >

Step 1 - Create a newNode with given value and set 'newNode → next' to NULL.

Step 2 - Check whether queue is Empty (rear == NULL)

Step 3 - If it is Empty then, set front = newNode and rear = newNode.

Step 4 - If it is Not Empty then, set rear → next = newNode and rear = newNode.

<Psuedo Code For Deletion From Front Of Queue>

Step 1 - Check whether queue is Empty (front == NULL).

Step 2 - If it is Empty, then display "Queue is Empty!!! Deletion is not possible!!!" and terminate from the function

Step 3 - If it is Not Empty then, define a Node pointer 'temp' and set it to 'front'.

Step 4 - Then set 'front = front → next' and delete 'temp' (free(temp)).

<Psuedo Code To Display The Queue>

Step 1 - Check whether queue is Empty (front == NULL).

Step 2 - If it is Empty then, display 'Queue is Empty!!!' and terminate the function.

Step 3 - If it is Not Empty then, define a Node pointer 'temp' and initialize with front.

Step 4 - Display 'temp → data --->' and move it to the next node. Repeat the same until 'temp' reaches to 'rear' (temp → next != NULL).

Step 5 - Finally! Display 'temp → data ---> NULL'.

Code For Implementation Of Queue Using Linked List: -

```
#include<iostream>
using namespace std;

class Node{
        public:
                int data;
                Node* next;
                Node(int CurrData){
                        this->data=CurrData;       this->next=NULL;
                }
};

class Queues_LinkedList{
   private:
      Node* Front , Rear;
      int size;
   public:
        Queues_LinkedList(int c){
                this->Front=this->Rear=NULL;    this->size=0;
        }
        void enqueue(int data){
                Node* NewNode=new Node(data);
                if(Rear==NULL){
                        Front=Rear=NewNode;    size++;         return;
                }
```

```
        Rear->next=NewNode;    Rear=NewNode;
        size++;
    }
    int dequeue(){
        if (Front==NULL){
            cout<<"QUEUE UNDERFLOW ::: QUEUE IS EMPTY"<<endl;
            return (-1);
        }
        int Value=Front->data;          Node *temp = Front;
        Front = Front ->next ;          temp->next=NULL;
        delete temp;                    return Value;
    }
    void PrintQueue(){
        cout<<"THE QUEUE IS ::: (FRONT)--> ";
        Node* temp=Front;
        while(temp->next!=NULL){
            cout<<temp->data<<" ";
            temp=temp->next;
        }
        cout<<temp->data<<" <--(REAR)"<<endl;
    }
};
```

```
int main(){
        Queues_LinkedList qll(10);
        for(int i = 1 ; i <= 5 ; i++){
                qll.enqueue(10*i);
        }
        qll.PrintQueue();
        qll.dequeue();      qll.PrintQueue();
        return 0;
}
```

OUTPUT : -

PROGRAM - 09.

Page No. 1.
Date 21 | 11 | 23

⇒ Viva Questions of Program - 09.

Q→ In Linked List Implementation of Queue, If only front pointer is been maintained, which Operation will Take Worst Case O(n) :-

(A) Insersion.
(B) Deletion.
(C) To Empty a Queue.
(D) Both Insersion and To Empty a Queue.

Q→ In Linked List Implementation of Queue, front and rear pointers are been Tracked, Then, which of The pointer will change during Insersion of Element in an Non-Empty Queue :-

(A) Front.
(B) Rear.
(C) Both Front & Rear.
(D) None of The Above.

Q→ In Linked List Implementation of a Queue, front and rear pointers are been Tracked. Which of The pointer will change during Insersion of Element in an Empty Queue :-

(A) Front.
(B) Rear.
(C) Both Front and Rear.
(D) None of The Above.

Q → In Linked List Implementation of Queue, from where the Element from the Queue is been deleted :-

(A) At the Head of List.
(B) At the Centre Position of List.
(C) At the Tail of List.
(D) Node Before the Tail.

Q → In Linked List Implementation of Queue, the Important Condition when Queue is Empty is :-

(A) Front is NULL.
(B) Rear is NULL.
(C) Link is Empty.
(D) Front == Rear - 1.

Program-10.

Q :- WAP To Create a Binary Tree using linked list & Display using Graphics and perform the following operations: Tree traversals (Preorder, Postorder, Inorder) using the concept of recursion?

< Psuedo Code For PreOrder Traversal Of Binary Tree>

```
void preorder(struct node *root) {
        if(root == NULL)        return;
        cout << root -> data << " ";
        preorder (root -> left);
        preorder(root -> right);
}
```

< Psuedo Code For InOrder Traversal Of Binary Tree>

```
void inorder(struct node *root) {
        if(root == NULL)        return;
        inorder (root -> left);
        cout << root -> data << " ";
        inorder(root -> right);
}
```

< Psuedo Code For InOrder Traversal Of Binary Tree>

```
void postorder(struct node *root) {
        if(root == NULL)        return;
        postorder (root -> left);
        postorder(root -> right);
        cout << root -> data << " ";
}
```

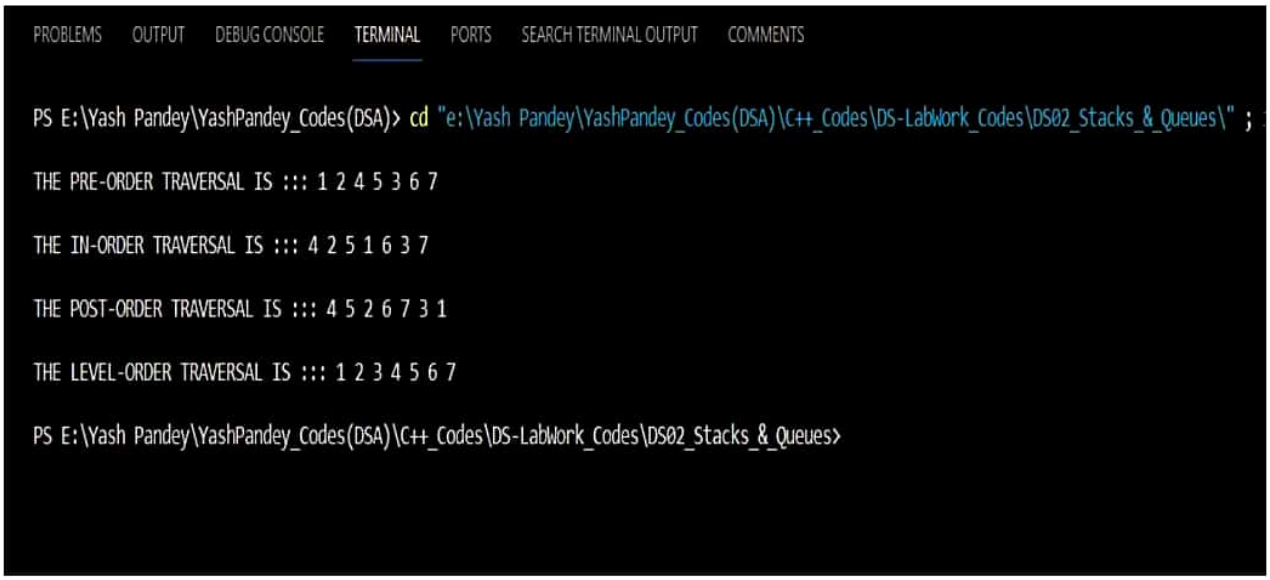
Code For Implementation Of Binary Tree Using Linked List: -

```
#include<bits/stdc++.h>
using namespace std;

class Node{
    public:
        int data;
        Node* Left , Right;
        Node(int CurrData){
            this->data=CurrData;
            this->Left=this->Right=NULL;
        }
};
void PreOrder_Traversal(Node* Root){
    if(Root==NULL){    return;    }
    cout<<Root->data<<" ";
    PreOrder_Traversal(Root->Left);
    PreOrder_Traversal(Root->Right);
}
void InOrder_Traversal(Node* Root){
    if(Root==NULL){    return;    }
    InOrder_Traversal(Root->Left);
    cout<<Root->data<<" ";
    InOrder_Traversal(Root->Right);
}
```

```
void PostOrder_Traversal(Node* Root){
        if(Root==NULL){      return;              }
        PostOrder_Traversal(Root->Left);
        PostOrder_Traversal(Root->Right);
        cout<<Root->data<<" ";
}
void LevelOrder_Traversal(Node* Root){
        if(Root==NULL){      return;              }
        queue<Node*> q;          // Inbuilt Queue In C++.....
        q.push(Root);       q.push(NULL);
        while( !q.empty() ){
                Node* CurrNode=q.front();       q.pop();
                if(CurrNode != NULL){
                        cout<<CurrNode->data<<" ";
                        if(CurrNode->Left != NULL){  q.push(CurrNode->Left);  }
                        if(CurrNode->Right != NULL){  q.push(CurrNode->Right);  }
                } else if( !q.empty() ){    q.push(NULL);       }
        }
}
int main(){
        Node* NewNode=new Node(1);
        NewNode->Left=new Node(2);          NewNode->Right=new Node(3);
        NewNode->Left->Left=new Node(4);
        NewNode->Left->Right=new Node(5);
        NewNode->Right->Left=new Node(6);
        NewNode->Right->Right=new Node(7);
```

```
        cout<<"THE PRE-ORDER TRAVERSAL IS ::: ";
        PreOrder_Traversal(NewNode); cout<<endl;
        cout<<"THE IN-ORDER TRAVERSAL IS ::: ";
        InOrder_Traversal(NewNode);          cout<<endl;
        cout<<"THE POST-ORDER TRAVERSAL IS ::: ";
        PostOrder_Traversal(NewNode);        cout<<endl;
        cout<<"THE LEVEL-ORDER TRAVERSAL IS ::: ";
        LevelOrder_Traversal(NewNode);       cout<<endl;
}
```

OUTPUT : -

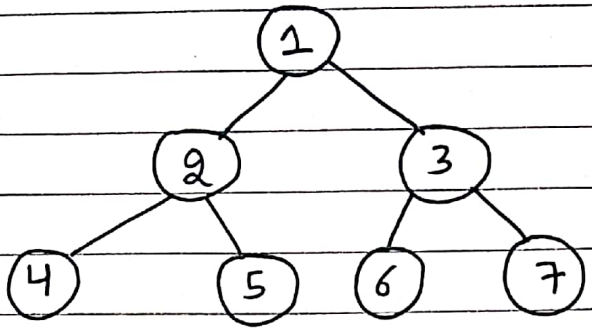


PROGRAM-10.

Page No. 1.
Date 21 | 11 | 23

→ Viva - Questions of Program - 10.

Q→



1. Height of Given Binary Tree ⟶ Height = 3

2. Preorder of Given Tree ⟶ 1, 2, 4, 5, 3, 6, 7

3. Inorder of Given Tree ⟶ 4, 2, 5, 1, 6, 3, 7

4. Postorder of Given Tree ⟶ 4, 5, 2, 6, 7, 3, 1

5. Levelorder of Given Tree ⟶ 1, 2, 3, 4, 5, 6, 7

Q→ How many child Nodes there can be for any parent Node in BT ?

→ There can only be 2 child for a parent Node in a Binary Tree which will Include one left child and one Right child.

Q→ How can You Traverse a Binary Tree ?

→ We can Traverse an Binary Tree Recursively & Iteratively.

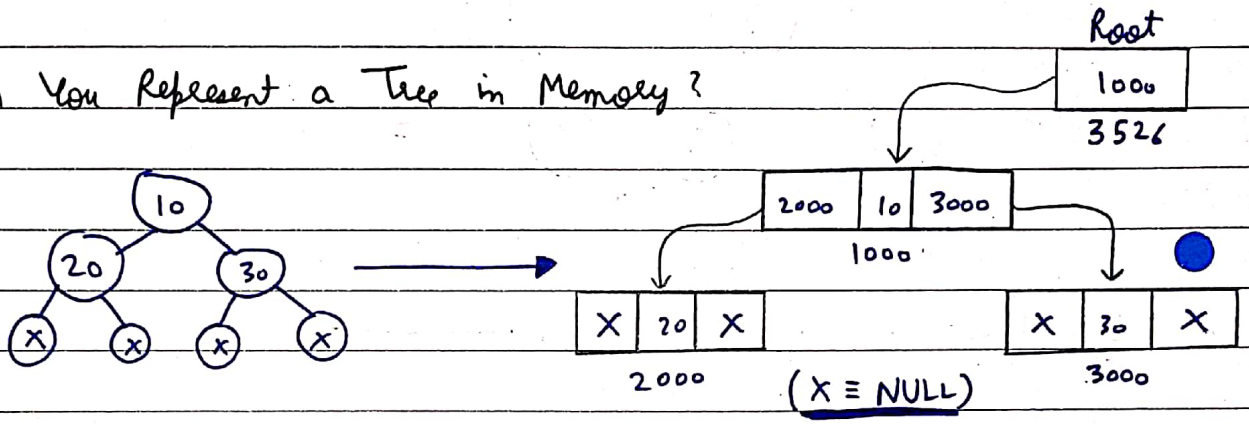
→ 4 Types of Traversal on a Binary Tree are Preorder, Inorder, Postorder and Level Order Traversal.

Q → Does The last Node of postorder Traversal and The First Node of The preorder of a Binary Tree are always Same ?

→ Yes, Both are Same as They Represent Root Node of Binary Tree.

Q → Can You Represent a Tree in Memory ?

→



Q → What is The Order of Nodes in Inorder Traversal ?

→ Order of Nodes ∴∴ Left – Root – Right , In Inorder Traversal....

Q → Can a Tree can be called as Graph also ?

→ Yes, Every Tree is an Bipartite Graph. A Graph is an Bipartite Graph iff It Doesn't Contains any Cycles of odd length. Since, a Tree has No Cycles, Thus, It is an Bipartite Graph.

∴ Thus, Every Tree Can also be called as Bipartite Graph.

Program-11.

Q :- WAP To Create a Binary Search Tree using linked list & Display using Graphics and perform the following operations: Tree traversals (Preorder, Postorder, Inorder) , Insersion & Deletion In BST?

< Psuedo Code For Insersion In Binary Search Tree>

Step 1: Create a newNode with given value and set its left and right to NULL.

Step 2: Check whether tree is Empty.

Step 3: If the tree is Empty, then set set root to newNode.

Step 4: If the tree is Not Empty, then check whether value of newNode is smaller or larger than the node (here it is root node).

Step 5: If newNode is smaller than or equal to the node, then move to left child. If newNode is larger than the node, then move to its right child.

Step 6: Repeat the above step until we reach to a leaf node (i.e., reach to NULL).

Step 7: After reaching a leaf node, then insert the newNode as left child if newNode is smaller or equal to that leaf else insert it as right child.

< Psuedo Code For Deletion In Binary Search Tree>

Case 1: Deleting a Leaf node (A node with no children)

Step 1: Find the node to be deleted using search operation

Step 2: Delete the node using free function (If it is a leaf) and terminate the function.

Case 2: Deleting a node with one child

Step 1: Find the node to be deleted using search operation

Step 2: If it has only one child, then create a link between its parent and child nodes.

Step 3: Delete the node using free function and terminate the function.

Case 3: Deleting a node with two children

Step 1: Find the node to be deleted using search operation

Step 2: If it has two children, then find the largest node in its left subtree (OR) the smallest node in its right subtree.

Step 3: Swap both deleting node and node which found in above step.

Step 4: Then, check whether deleting node came to case 1 or case 2 else goto steps 2

Step 5: If it comes to case 1, then delete using case 1 logic.

Step 6: If it comes to case 2, then delete using case 2 logic.

Step 7: Repeat the same process until node is deleted from the tree.

Code For Implementation Of BST Using Linked List: -

```
#include<iostream>
using namespace std;

class Node{
        public:
                int data;       Node* Left ,  Right;
                Node(int CurrData){
                        this->data=CurrData;
                        this->Left=this->Right=NULL;
                }
};
Node* Build_BST(Node* Root , int Value){
        if(Root==NULL){    Root=new Node(Value);     return Root;    }
        else if(Root->data>Value){   Root->Left=Build_BST(Root->Left,Value);   }
      else if(Root->data<Value){   Root->Right=Build_BST(Root->Right,Value );   }
```

```
        return Root;
}
void PreOrder_Traversal(Node* Root){
        if(Root==NULL){         return;     }
        cout<<Root->data<<" ";
        PreOrder_Traversal(Root->Left);
        PreOrder_Traversal(Root->Right);
}
void InOrder_Traversal(Node* Root){
        if(Root==NULL){         return;     }
        InOrder_Traversal(Root->Left);
        cout<<Root->data<<" ";
        InOrder_Traversal(Root->Right);
}
void PostOrder_Traversal(Node* Root){
        if(Root==NULL){         return;     }
        PostOrder_Traversal(Root->Left);
        PostOrder_Traversal(Root->Right);
        cout<<Root->data<<" ";
}
Node* Find_InOrder_Sucsessor(Node* Root){
        Node* Temp=Root;
        while(Root->Left != NULL){      Temp=Temp->Left;        }
        return Temp;
}
```

```
Node* Delete_In_BST(Node* Root, int Value) {
        if (Root->data > Value) {
                Root->Left = Delete_In_BST(Root->Left, Value);
        }
        else if (Root->data < Value) {
                Root->Right = Delete_In_BST(Root->Right, Value);
        }
    else {
         //Case 1--> When The Target Node Is A Leaf Node(Has No Child).....
        if (Root->Left == NULL && Root->Right == NULL) {  return NULL;     }

        //Case 2--> When The Target Node Has One Child.....
        else if (Root->Left == NULL) {     return Root->Right;          }
        else if (Root->Right == NULL) {  return Root->Left;           }

        //Case 3--> When The Target Node Has Two Child.....
        Node* InOrdSucs = Find_InOrder_Sucsessor(Root->Right);
        Root->data = InOrdSucs->data;
        Root->Right = Delete_In_BST(Root->Right, InOrdSucs->data);
    }
    return Root;
}
int main(){
        Node* NewNode=NULL;
        for(int i = 1 ; i <= 7 ; i++){          NewNode=Build_BST(NewNode,i);     }
        cout<<"THE PRE-ORDER TRAVERSAL IS ::: ";
```

```
    PreOrder_Traversal(NewNode);            cout<<endl;
    cout<<"THE IN-ORDER TRAVERSAL IS ::: ";
    InOrder_Traversal(NewNode);             cout<<endl;
    cout<<"THE POST-ORDER TRAVERSAL IS ::: ";
    PostOrder_Traversal(NewNode);           cout<<endl;
    cout<<"Node With Value=6 Is Deleted From The BST....."<<endl;
    Delete_In_BST(NewNode,6);
    cout<<"THE PRE-ORDER TRAVERSAL IS ::: ";
    PreOrder_Traversal(NewNode);            cout<<endl;
    cout<<"THE IN-ORDER TRAVERSAL IS ::: ";
    InOrder_Traversal(NewNode);             cout<<endl;
    cout<<"THE POST-ORDER TRAVERSAL IS ::: ";
    PostOrder_Traversal(NewNode);           cout<<endl;
    return 0;
}
```

OUTPUT : -

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SEARCH TERMINAL OUTPUT  COMMENTS

PS E:\Yash Pandey\YashPandey_Codes(DSA)> cd "e:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS02_Stacks_&_Queues\" ;

THE PRE-ORDER TRAVERSAL IS ::: 1 2 3 4 5 6 7
THE IN-ORDER TRAVERSAL IS ::: 1 2 3 4 5 6 7
THE POST-ORDER TRAVERSAL IS ::: 7 6 5 4 3 2 1

Node With Value=6 Is Deleted From The BST.....

THE PRE-ORDER TRAVERSAL IS ::: 1 2 3 4 5 7
THE IN-ORDER TRAVERSAL IS ::: 1 2 3 4 5 7
THE POST-ORDER TRAVERSAL IS ::: 7 5 4 3 2 1

PS E:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS02_Stacks_&_Queues>
```

PROGRAM-11.

⇒ Viva - Questions of Program - 11.

Q→ What is The Speciality about The Inorder Traversal of BST :-

(A) It Traverses in Non - Increasing Order.
(B) It Traverses in Increasing Order.
(C) It Traverses in Random fashion.
(D) It Traverses Based on The Priority of Nodes.

Q→ What is The Worst Case & Average Case Complexity of Search in BST :-

(A) O(n), O(n).
(B) O(log n), O(log n).
(C) O(n), O(log n).
(D) O(1), O(1).

Q→ What is The Condition of Optimal BST and It's Advantage :-

(A) The Tree should not be modified and you should know how often the Keys are Accessed and Improves The lookup Cost.
(B) You Should Know the frequency of Access of Keys & Improves lookup Time.
(C) The Tree can be modified and you Should Know The N.O. of Elements in The Tree before hand as it Improves The Deletion Time.
(D) The Tree Should be Just modified and improves the lookup Time.

Q→ What Does below Code does :-

```
public void func ( Tree Root) {
        Cout << Root → data << Endl;
        func (Root → left);
        func (Root → Right);
}
```

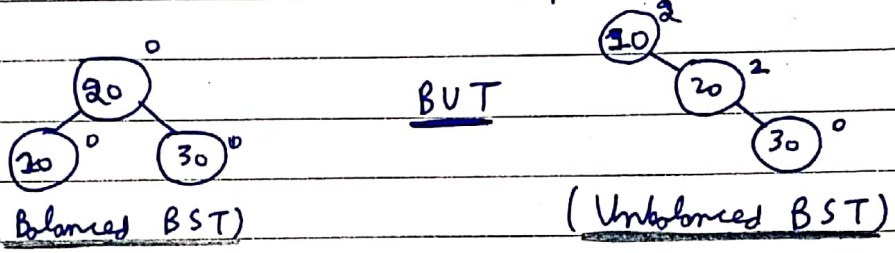
This Given Code will perform Preorder Traversal of The Given Binary Tree.....

Q → What are The Advantages of BST ?

→ Efficient Insersion and Deletion in BST.
→ Fast Search (O(log n)) in an BST.
→ Simplified Implementation and Dynamic Sizing.

Q → Whether a BST is an Balanced Binary Tree ?

→ Yes, In Some Cases, BST Can be an Balanced Tree but It Can also be an Unbalanced Tree. It depends on The Structure of BST.



(Balanced BST) (Unbalanced BST)

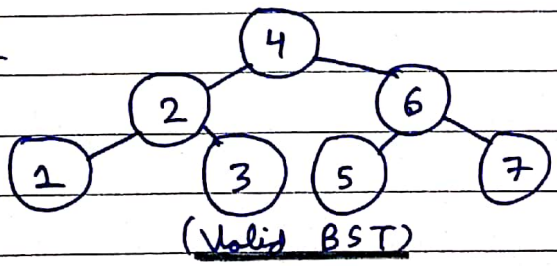
Q → Which Traversal on BST will Generate an Ascending order list of Nodes?

→ Inorder Traversal of BST will Generate an Ascending Order list.

Q → How will You Decide The Root of BST ?

→ As BST follows a Special Property That Nodes To Left Subtree are lesser Than The Root Node and Nodes in Right Subtree must be Greater Than The Root Node. Thus, We have To Decide Root Node Such That Nodes To left of Root Node must be lesser Than Root Node and The Nodes of Right Subtree must be Greater Than Root Node.

◉ Eg:-



(Valid BST)

→ Inorder = 1, 2, 3, 4, 5, 6, 7.

Program-12.

Q :- WAP To Implement Sorting techniques using array. (Insertion sort, Merge sort, Quick sort, Bubble sort, Bucket sort, Radix sort, Shell sort, Selection sort, Heap sort and Exchange sort).

1. Bubble Sort Algorithm

< Psuedo Code For Bubble Sort Algorithm>

```
void BubbleSort(int arr[],int n){
        for(int i=0 ; i < n ; i++){
                for(int j=0 ; j<(n-1-i) ; j++){
                        if(arr[j]>arr[j+1]){        /* Swap arr[i] And arr[j] */        }
                }
        }
}
```

Code For Implementation Of Bubble Sort Algorithm: -

```
#include<iostream>
using namespace std;
void PrintArray(int*,int);
void BubbleSort(int arr[ ] , int n){      //O(n^2).....
        for(int i = 0  ;  i < n ; i++){
                for(int j=0 ;  j < (n-1-i) ; j++){
                        if(arr[j]>arr[j+1]){
                                int Temp=arr[j];      arr[j]=arr[j+1];       arr[j+1]=Temp;
                        }
        }
```

```
    }
    cout<<"After Bubble Sort(In Ascending Order) --> ";
    PrintArray(arr,n);
}
void PrintArray(int arr[] , int n){
        cout<<"THE ARRAY IS ::: { ";
        for(int i =0 ; i < n ; i++){
                if(i == (n-1)){         cout<<arr[i]<<" }"<<endl;         }
                else{  cout<<arr[i]<<", ";          }
        }
        cout<<endl;
}

int main(){
        int arr[8]={3,2,6,1,5,7,8,4};
        PrintArray(arr,8);          BubbleSort(arr,8);
        return 0;
}
```

OUTPUT : -

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SEARCH TERMINAL OUTPUT   COMMENTS

PS E:\Yash Pandey\YashPandey_Codes(DSA)> cd "e:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS06_Sorting_Methods\" ;
THE ARRAY IS ::: { 3, 2, 6, 1, 5, 7, 8, 4 }

After Bubble Sort(In Ascending Order) --> THE ARRAY IS ::: { 1, 2, 3, 4, 5, 6, 7, 8 }

PS E:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS06_Sorting_Methods>
```

2. Selection Sort Algorithm

< Psuedo Code For Selection Sort Algorithm>

```
void SelectionSort(int arr[],int n){
        for(int i=0 ; i < (n-1) ; i++){
                int max=i;
                for(int j=(i+1) ; j < n ; j++){
                        if(arr[max] > arr[j]){    max=j;  }
                }
                //Swap arr[i] And arr[max].....
        }
}
```

Code For Implementation Of Selection Sort Algorithm: -

```
#include<iostream>
using namespace std;
void PrintArray(int*,int);
void SelectionSort(int arr[],int n){      //O(n^2).....
        for(int i=0 ; i < (n-1) ; i++){
                int max=i;
                for(int j = (i+1) ; j < n ; j++){
                        if(arr[max] > arr[j]){    max=j;  }
                }
                int Temp=arr[i];     arr[i]=arr[max];     arr[max]=Temp;
        }
        cout<<"After Selection Sort(In Ascending Order) --> ";
        PrintArray(arr,n);
}
```

```
void PrintArray(int arr[ ] , int n){
        cout<<"THE ARRAY IS ::: { ";
        for(int i = 0 ; i < n ; i++){
                if(i == (n-1)){         cout<<arr[i]<<" }"<<endl; }
                else{           cout<<arr[i]<<", ";                         }
        }
        cout<<endl;
}
int main(){
        int arr[8]={3,2,6,1,5,7,8,4};
        PrintArray(arr,8);
        SelectionSort(arr,8);
        return 0;
}
```

OUTPUT : -

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SEARCH TERMINAL OUTPUT   COMMENTS

PS E:\Yash Pandey\YashPandey_Codes(DSA)> cd "e:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS06_Sorting_Methods\" ;
THE ARRAY IS ::: { 3, 2, 6, 1, 5, 7, 8, 4 }

After Selection Sort(In Ascending Order) --> THE ARRAY IS ::: { 1, 2, 3, 4, 5, 6, 7, 8 }

PS E:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS06_Sorting_Methods>
```

3. Insertion Sort Algorithm

< Psuedo Code For Insertion Sort Algorithm>

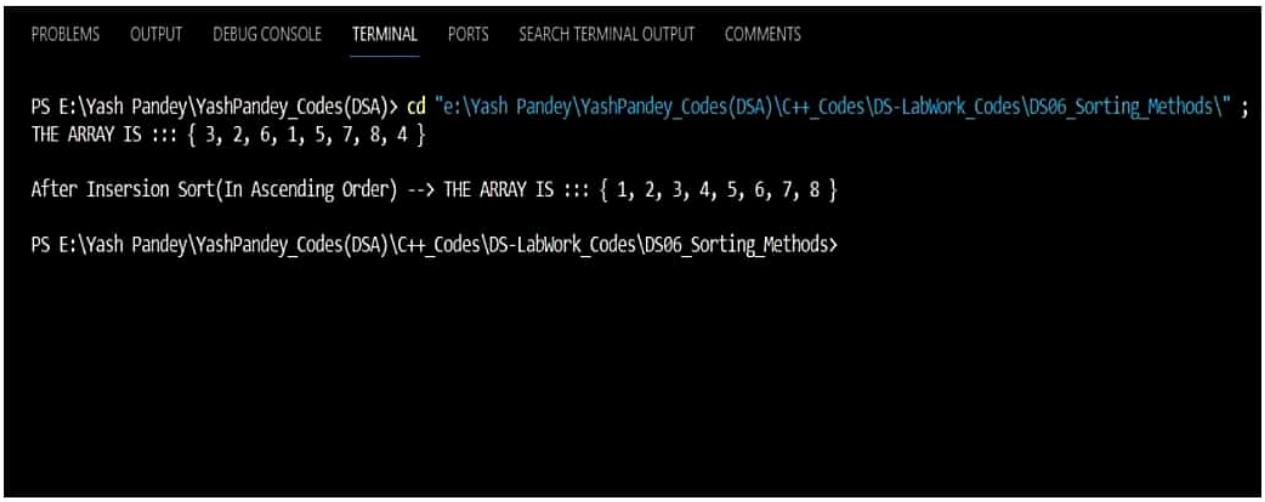
```
void InsertionSort(int arr[],int n){
        for(int i=1 ; i < n ; i++){
                int Curr=arr[i] , prev=i-1;
                while(prev>=0 && arr[prev]>Curr){
                        arr[prev+1]=arr[prev];   prev--;
                }
                arr[prev+1]=Curr;
        }
}
```

Code For Implementation Of Insertion Sort Algorithm: -

```
#include<iostream>
using namespace std;
void PrintArray(int*,int);
void InsertionSort(int arr[] , int n){      //O(n^2).....
        for(int i=1 ; i < n ; i++){
                int Curr=arr[i];        int prev=i-1;
                while(prev >= 0 && arr[prev] > Curr){
                        arr[prev+1]=arr[prev];      prev--;
                }
                arr[prev+1]=Curr;
        }
        cout<<"After Insersion Sort(In Ascending Order) --> ";
        PrintArray(arr,n);
}
```

```
void PrintArray(int arr[],int n){
        cout<<"THE ARRAY IS ::: { ";
        for(int i =0 ; i < n ; i++){
                if(i == (n-1)){                 cout<<arr[i]<<" }"<<endl; }
                else{           cout<<arr[i]<<", ";          }
        }
        cout<<endl;
}
int main(){
        int arr[8]={3,2,6,1,5,7,8,4};
        PrintArray(arr,8);
        InsertionSort(arr,8);
        return 0;
}
```

OUTPUT : -



4. Bucket Sort Algorithm

< Psuedo Code For Bucket Sort Algorithm>

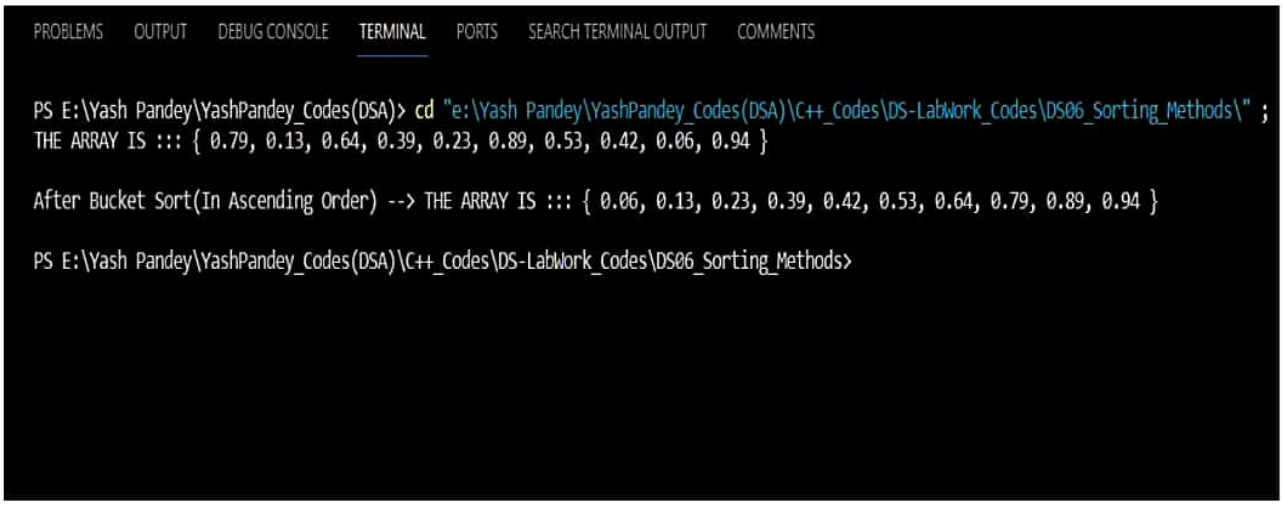
```
void BucketSort(float arr[],float sorted[],int n){
        for(int i=0;i<n;i++){
                int idx=arr[i]*n;
                sorted[idx]=arr[i];
        }
}
```

Code For Implementation Of Bucket Sort Algorithm: -

```
#include<iostream>
using namespace std;
void PrintArray(float*,int);

void BucketSort(float arr[] , float sorted[] , int n){          //O(n)......
        for(int i = 0 ; i < n ; i++){
                int idx=arr[i]*n;      sorted[idx]=arr[i];
        }
        cout<<"After Bucket Sort(In Ascending Order) --> ";
        PrintArray(sorted,n);
}
void PrintArray(float arr[],int n){
        cout<<"THE ARRAY IS ::: { ";
        for(int i = 0 ; i < n ; i++){
                if(i==(n-1)){          cout<<arr[i]<<" }"<<endl;}
                else{          cout<<arr[i]<<", ";          }
        }
```

```
        cout<<endl;
}
int main(){
        float arr[10]={0.79,0.13,0.64,0.39,0.23,0.89,0.53,0.42,0.06,0.94};
        float sorted[10]={0};
        PrintArray(arr,10);
        BucketSort(arr,sorted,10);
        return 0;
}
```

OUTPUT : -



5. Radix Sort Algorithm

< Psuedo Code For Radix Sort Algorithm>

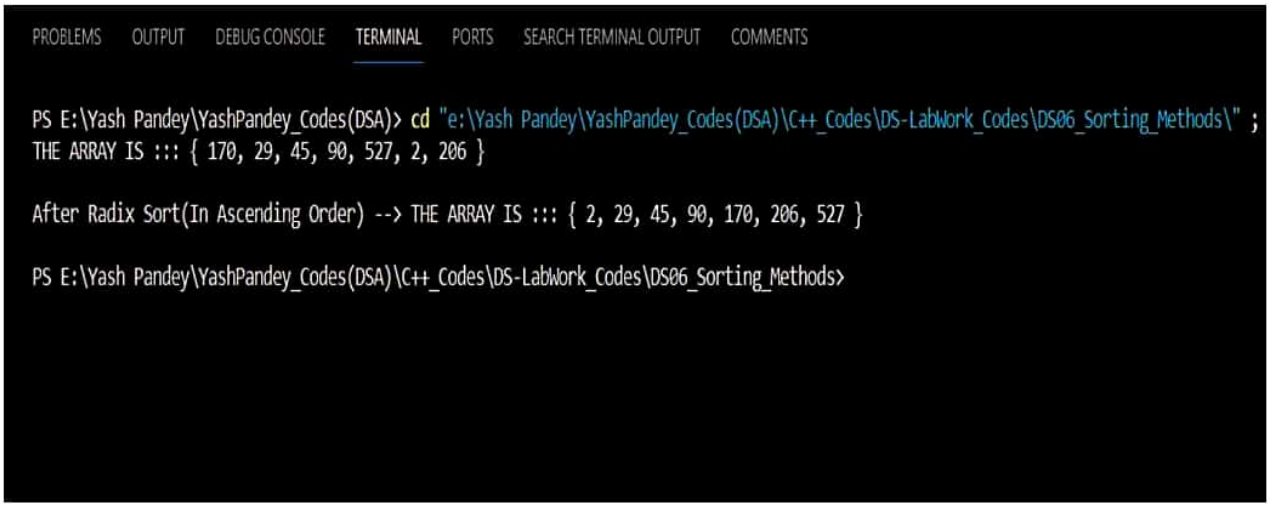
```
void RadixSort(int arr[],int n){
    int MaxElement=GetMax(arr,n);
    for(int pos=1 ; (MaxElement/pos)>0 ;pos*=10){    CountSort(arr,pos,n);    }
}
```

Code For Implementation Of Radix Sort Algorithm: -

```
#include<iostream>
using namespace std;
void PrintArray(int*,int);
int GetMax(int arr[],int n){
    int Max = -1;
    for(int i=0;i<n;i++){        if(arr[i]>Max){ Max=arr[i]; }        }
    return Max;
}
void CountSort(int arr[],int pos,int n){
    int freq[10]={0};
    for(int i = 0 ; i < n ; i++){        freq[(arr[i]/pos)%10]++;        }
    for(int i = 1 ; i < 10 ; i++){       freq[i] + = freq[i-1];          }
    int output[n]={0};
    for(int i = (n-1) ; i >= 0 ; i--){   output[--freq[(arr[i]/pos)%10]] = arr[i];   }
    for(int i = 0 ; i < n ; i++){        arr[i] = output[i];        }
}

void RadixSort(int arr[],int n){
    int MaxElement=GetMax(arr,n);
```

```
        for(int pos = 1;(MaxElement/pos) > 0 ;pos*=10){  CountSort(arr,pos,n); }
        cout<<"After Radix Sort(In Ascending Order) --> ";
        PrintArray(arr,n);
}
void PrintArray(int arr[],int n){
        cout<<"THE ARRAY IS ::: { ";
        for(int i = 0 ; i < n ; i++){
                if(i==(n-1)){          cout<<arr[i]<<" }"<<endl;}
                else{          cout<<arr[i]<<", ";          }
        }  cout<<endl;
}
int main(){
   int arr[7]={170,29,45,90,527,2,206};
   PrintArray(arr,7);        RadixSort(arr,7);
   return 0;
}
```

OUTPUT : -



6. Shell Sort Algorithm

< Psuedo Code For Shell Sort Algorithm>

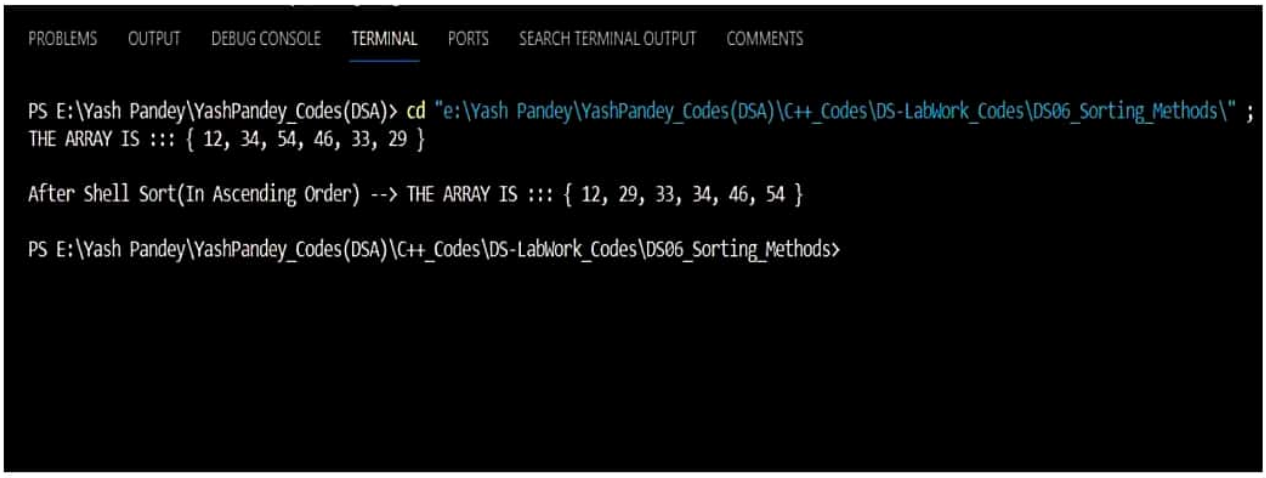
```
int ShellSort(int arr[], int n) {
        for (int gap = n/2; gap > 0; gap /= 2) {
                for (int i = gap; i < n; i += 1) {
                        int temp = arr[i], j;
                        for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {  arr[j] = arr[j - gap]; }
                        arr[j] = temp;
                }
        }
}
```

Code For Implementation Of Shell Sort Algorithm: -

```
#include <iostream>
using namespace std;
void PrintArray(int*,int);
int ShellSort(int arr[], int n) {
        for (int gap = n/2; gap > 0; gap /= 2) {
                for (int i = gap; i < n; i += 1) {
                        int temp = arr[i], j;
                        for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                                arr[j] = arr[j - gap];
                        }  arr[j] = temp;
                }
        }
        cout<<"After Shell Sort(In Ascending Order) --> ";
        PrintArray(arr,n);
}
```

```
void PrintArray(int arr[],int n){
        cout<<"THE ARRAY IS ::: { ";
        for(int i = 0 ; i < n ; i++){
                if(i == (n-1)){         cout<<arr[i]<<" }"<<endl;}
                else{           cout<<arr[i]<<", ";          }
        }
        cout<<endl;
}
int main() {
        int arr[6] = {12,34,54,46,33,29};
        PrintArray(arr,6);
        ShellSort(arr,6);
        return 0;
}
```

OUTPUT : -



7. Exchange Sort Algorithm

< Psuedo Code For Exchange Sort Algorithm>

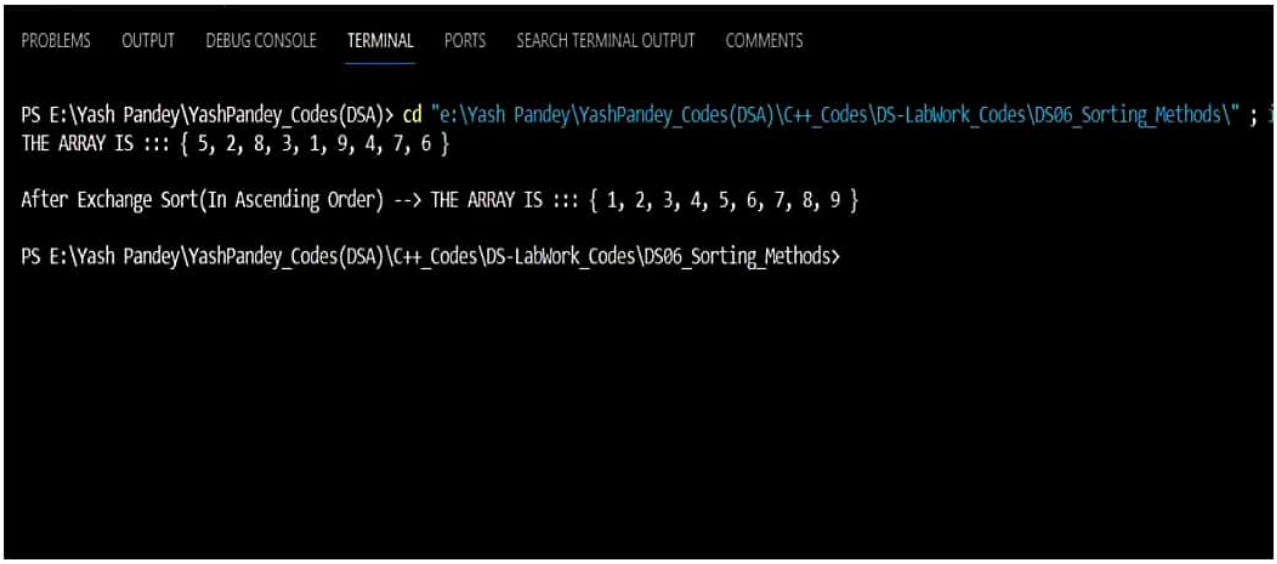
```
void ExchangeSort(int arr[], int n) {
        for (int i = 0; i < (n - 1); i++) {
            for (int j = 0; j < (n - i - 1); j++) {
                if (arr[j] > arr[j + 1]) {   /* swap arr[j] & arr[j+1]  */     }
            }
       }
}
```

Code For Implementation Of Exchange Sort Algorithm: -

```
#include <iostream>
using namespace std;
void PrintArray(int*,int);
void ExchangeSort(int arr[], int n) {
        for (int i = 0; i < (n - 1); i++) {
                for (int j = 0; j < (n - i - 1); j++) {
                        if (arr[j] > arr[j + 1]) {
                                int temp = arr[j];
                                arr[j] = arr[j + 1];    arr[j + 1] = temp;
                        }
                }
        }
        cout<<"After Exchange Sort(In Ascending Order) --> ";
        PrintArray(arr,n);
}
void PrintArray(int arr[] , int n){
```

```
        cout<<"THE ARRAY IS ::: { ";
        for(int i =0 ; i < n ; i++){
            if(i == (n-1)){         cout<<arr[i]<<" }"<<endl;         }
            else{  cout<<arr[i]<<", ";          }
        }
        cout<<endl;
}
int main() {
        int arr[9] = {5,2,8,3,1,9,4,7,6};
        PrintArray(arr,9);
        ExchangeSort(arr, 9);
        return 0;
}
```

OUTPUT : -



8. Merge Sort Algorithm

< Psuedo Code For Merge Sort Algorithm>

```
void MergeSort(int arr[],int Start,int End){
        if(Start>=End){     return;    }
        int Mid=Start+(End-Start)/2;
        MergeSort(arr,Start,Mid);     MergeSort(arr,Mid+1,End);
        Merge(arr,Start,Mid,End);
}
```

Code For Implementation Of Merge Sort Algorithm: -

```
#include<iostream>
using namespace std;
void Merge(int*,int,int,int);
void PrintArray(int*,int);
void MergeSort(int arr[],int Start,int End){          // O(nLogn).....
        if(Start>=End){      return;        }
        int Mid=Start+(End-Start)/2;
        MergeSort(arr,Start,Mid);        MergeSort(arr,Mid+1,End);
        Merge(arr,Start,Mid,End);
}
void Merge(int arr[],int Start,int Mid,int End){
        int Temp[End-Start+1]={0};
        int ptr1=Start , ptr2=Mid+1 , ptr=0;
        while(ptr1<=Mid  &&  ptr2<=End){
                if(arr[ptr1] <= arr[ptr2]){          Temp[ptr++] = arr[ptr1++];        }
                else {          Temp[ptr++] = arr[ptr2++];                }
        }
```

```
        while(ptr1 <= Mid){        Temp[ptr++] = arr[ptr1++];                }
        while(ptr2 <= End){        Temp[ptr++] = arr[ptr1++];                }
        int idx=Start;
        for(int i =0 ; i < (End-Start+1) ; i++ ){      arr[idx]=Temp[i];     idx++;      }
}
void PrintArray(int arr[] , int n){
        cout<<"THE ARRAY IS ::: { ";
        for(int i =0 ; i < n ; i++){
                if(i == (n-1)){          cout<<arr[i]<<" }"<<endl;          }
                else{  cout<<arr[i]<<", ";          }
        }       cout<<endl;
}
int main(){
   int arr[8]={3,2,6,1,5,7,8,4};
   PrintArray(arr,8);          MergeSort(arr,0,7);
   cout<<"After Merge Sort , ";          PrintArray(arr,8);
   return 0;
}
```

OUTPUT : -

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SEARCH TERMINAL OUTPUT  COMMENTS

PS E:\Yash Pandey\YashPandey_Codes(DSA)> cd "e:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS06_Sorting_Methods\" ;
THE ARRAY IS ::: { 3, 2, 6, 1, 5, 7, 8, 4 }

After Merge Sort , THE ARRAY IS ::: { 1, 2, 3, 4, 5, 6, 7, 8 }

PS E:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS06_Sorting_Methods>
```

9. Quick Sort Algorithm

< Psuedo Code For Quick Sort Algorithm>

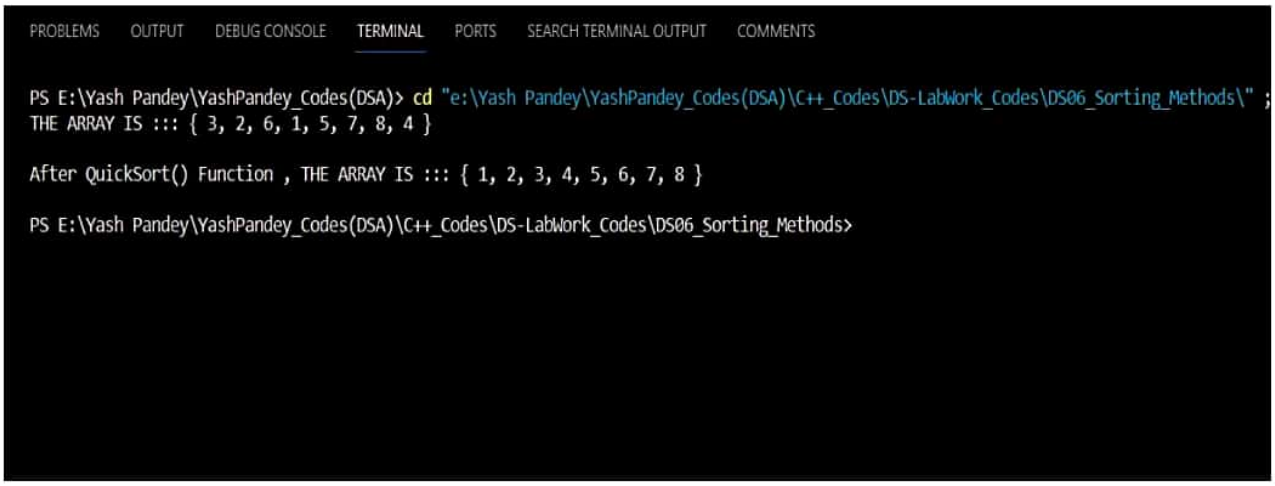
```
void QuickSort(int arr[],int Start,int End){  //O(nLogn).....
        if(Start>=End){     return;      }
        int Pivot=Partition(arr,Start,End);
        QuickSort(arr,Start,Pivot-1);
        QuickSort(arr,Pivot+1,End);
}
```

Code For Implementation Of Quick Sort Algorithm: -

```
#include<iostream>
using namespace std;
int Partition(int*,int,int);
void PrintArray(int*,int);
void QuickSort(int arr[],int Start,int End){          //O(nLogn).....
        if(Start>=End){      return;        }
        int Pivot=Partition(arr,Start,End);
        QuickSort(arr,Start,Pivot-1);
        QuickSort(arr,Pivot+1,End);
}
int Partition(int arr[],int Start,int End){
        int Pivot=arr[Start] , Count=0;
        for(int i = (Start+1) ; i <= End ; i++){
                if(arr[i] <= Pivot){          Count++;              }
}
```

```
        int PivotIndex=Start+Count , Temp=arr[PivotIndex];
        arr[PivotIndex]=arr[Start];         arr[Start]=Temp;
        int ptr1=Start , ptr2=End;
        while(ptr1<PivotIndex && ptr2>PivotIndex){
                while(arr[ptr1]<Pivot){    ptr1++;    }
                while(arr[ptr2]>Pivot){    ptr2--;    }
                if(ptr1<PivotIndex && ptr2>PivotIndex){
                        int Temp=arr[ptr1];    arr[ptr1]=arr[ptr2];   arr[ptr2]=Temp;
                        ptr1++;         ptr2--;
                }
        }       return PivotIndex;
}
void PrintArray(int arr[] , int n){
        cout<<"THE ARRAY IS ::: { ";
        for(int i =0 ; i < n ; i++){
                if(i == (n-1)){         cout<<arr[i]<<" }"<<endl;          }
                else{  cout<<arr[i]<<", ";          }
        }       cout<<endl;
}
int main(){
        int arr[8]={3,2,6,1,5,7,8,4};
        PrintArray(arr,8);    QuickSort(arr,0,7);
        cout<<"After QuickSort() Function , ";
        PrintArray(arr,8);
        return 0;
}
```

OUTPUT : -



10. Heap Sort Algorithm

< Psuedo Code For Heap Sort Algorithm>

```
void HeapSort(int arr[],int Size) {
        int n = Size;
        for (int i = (n / 2); i >= 0; i--) {      Heapify(arr, i, n);      }
       for (int i = (n - 1); i > 0; i--) {
                    /* Swap arr[0] And arr[i] */
                     Heapify(arr, 0, i);
        }
}
```

Code For Implementation Of Heap Sort Algorithm: -

```
#include<iostream>
using namespace std;
void Heapify(int*,int,int);
void HeapSort(int arr[],int Size) {       // O(nLogn).....
        int n = Size;
        for (int i = (n / 2); i >= 0; i--) {            Heapify(arr, i, n);            }
        for (int i = (n - 1); i > 0; i--) {
                int temp = arr[0];            arr[0] = arr[i];
                arr[i] = temp;                Heapify(arr, 0, i);
        }
}
void Heapify(int arr[], int i, int size) {          // O(Logn).....
        int Left = (2 * i) + 1;
        int Right = (2 * i) + 2;
        int MaxIDX = i;
        if (Left < size && arr[Left] > arr[MaxIDX]) {       MaxIDX = Left;       }
```

```
        if (Right < size && arr[Right] > arr[MaxIDX]) {          MaxIDX = Right;      }
        if (MaxIDX != i) {
                int temp = arr[i];              arr[i] = arr[MaxIDX];
                arr[MaxIDX] = temp;         Heapify(arr, MaxIDX, size);
        }
}
void PrintArray(int arr[] , int n){
        cout<<"THE ARRAY IS ::: { ";
        for(int i =0 ; i < n ; i++){
                if(i == (n-1)){         cout<<arr[i]<<" }"<<endl;          }
                else{           cout<<arr[i]<<", ";          }
        }       cout<<endl;
}
int main(){
        int arr[8]={3,2,6,1,5,7,8,4};
        PrintArray(arr,8);              HeapSort(arr,8);
        cout<<"After Heap Sort In Ascending Order , ";       PrintArray(arr,8);
        return 0;
}
```

OUTPUT : -

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SEARCH TERMINAL OUTPUT   COMMENTS

PS E:\Yash Pandey\YashPandey_Codes(DSA)> cd "e:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS06_Sorting_Methods\" ;
THE ARRAY IS ::: { 3, 2, 6, 1, 5, 7, 8, 4 }

After Heap Sort In Ascending Order , THE ARRAY IS ::: { 1, 2, 3, 4, 5, 6, 7, 8 }

PS E:\Yash Pandey\YashPandey_Codes(DSA)\C++_Codes\DS-LabWork_Codes\DS06_Sorting_Methods>
```

PROGRAM - 12.

Page No. 1.
Date 21 11 23

→ Viva Questions of Program - 12.

Q→ What is The Worst Case Time Complexity of Quick Sort?

→ The Worst Case Time Complexity of Quick Sort is $O(n^2)$ and This occurs when The Input Element Array is already sorted and The first is always been chosen as The Smallest Element of The Array. In This Case, The Algorithm will devolve a Simple Linear Search But with $O(n^2)$.

→ Average Case Time Complexity of Quick Sort is $O(n \log n)$.

Q→ What are The Types of Sorting?

→ There are only 2 Sorting i.e. Ascending Sorting & Descending Sorting.

Q→ Which Sorting Method is best?

→ Quick Sort is The Most Efficient Sorting Algorithm with $O(n \log n)$.

Q→ Solve The Given Array Using Insertion Sort?

→

29	20	73	34	64

Step I ::

20	29	73	34	64

Step II :::

20	29	34	73	64

Step III :::

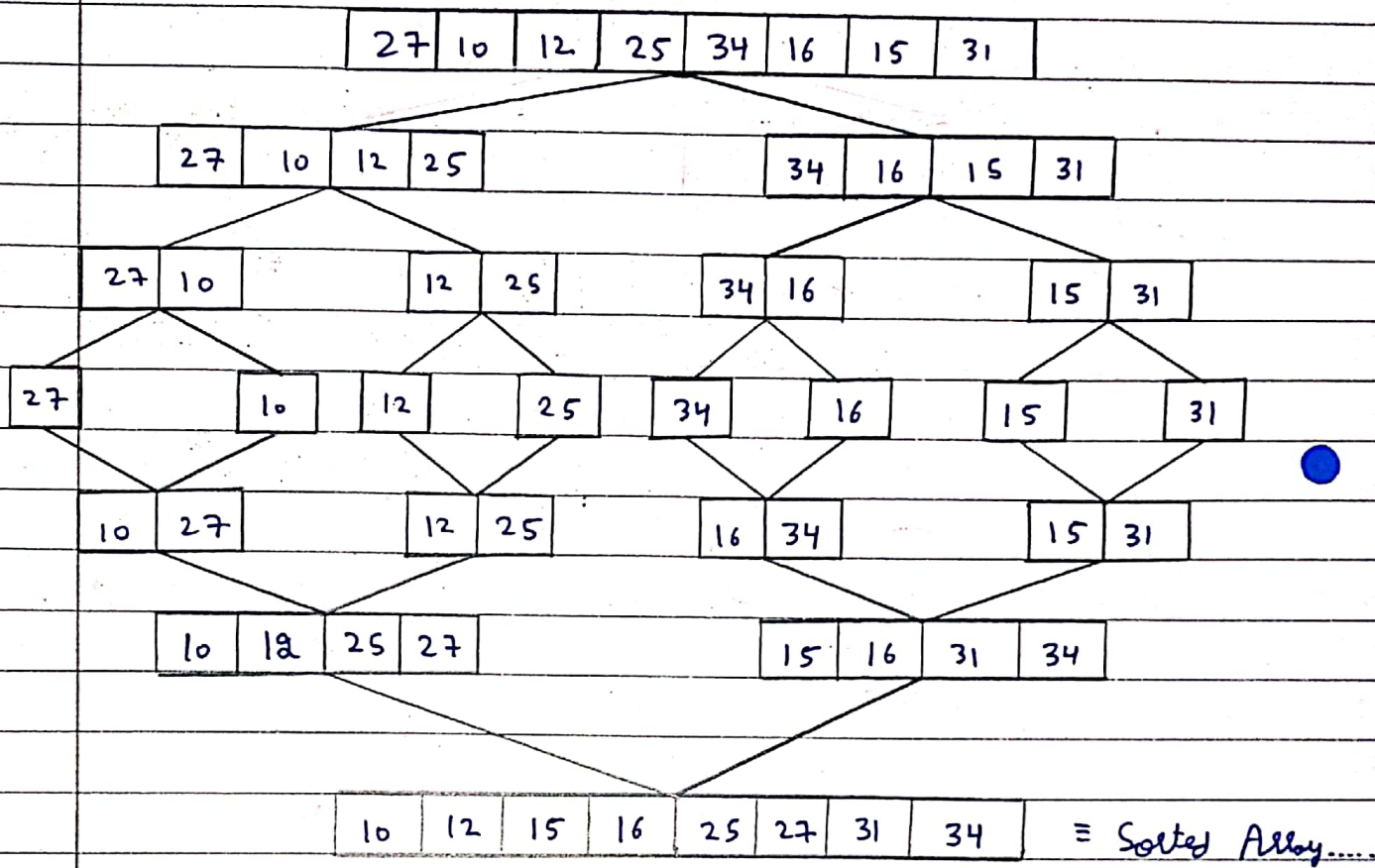
20	29	34	64	73

Step IV :::

20	29	34	64	73

→ Sorted Array.....

Q→ Consider The Given Array and Sort it using Merge Sort ?



∴ Thus, We Got Sorted Array with O(n log n) Time Complexity.